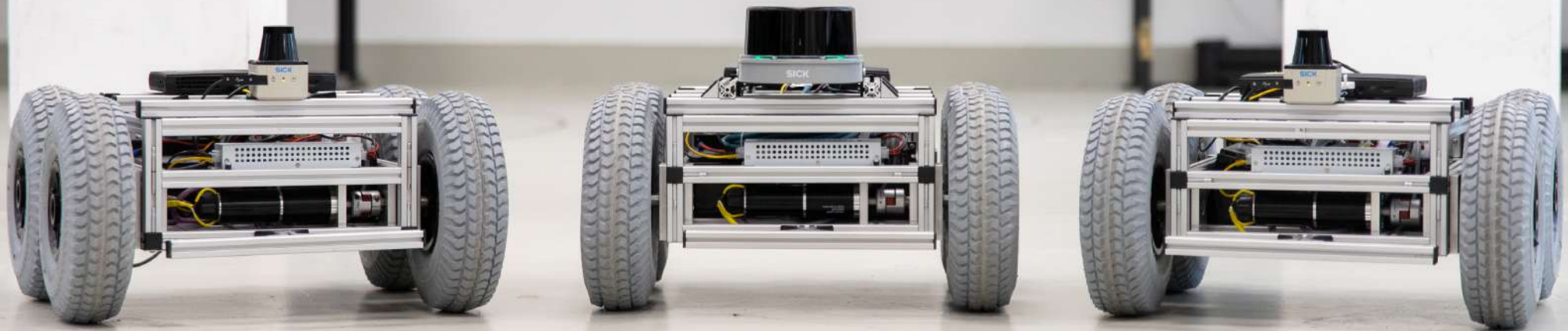


Robotik

Prof. Dr. Thomas Wiemann - FB AI - Robotik



Hochschule Fulda
University of Applied Sciences



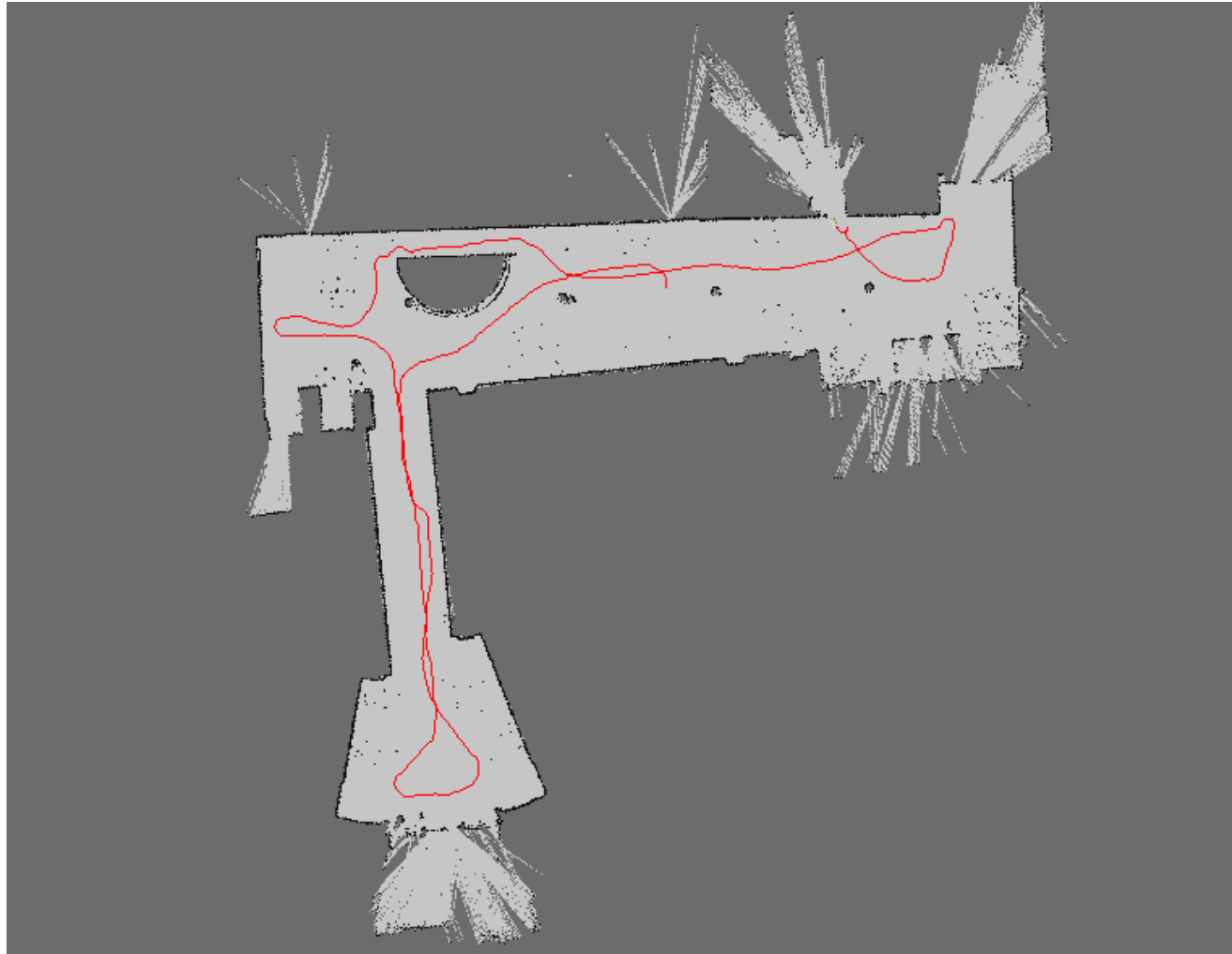


Gliederung

1. Einleitung
2. Robot Operating System
3. Sensordatenverarbeitung
4. Lokalisierung
- 5. Mapping**
6. Navigation
7. Ausblick



Wo kommt die Karte her?



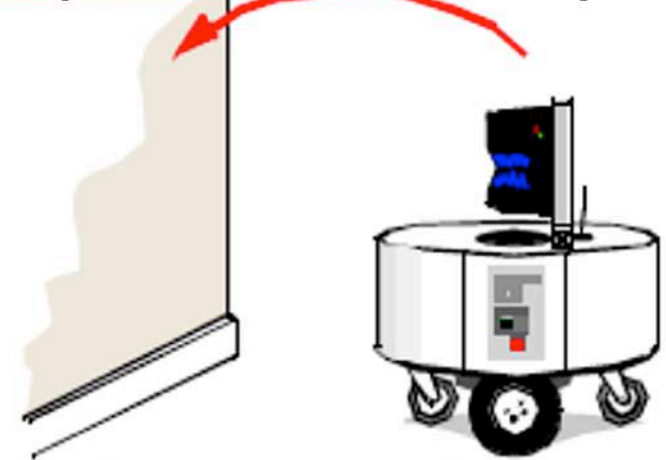


- ▶ Problembeschreibung:
 - Ausgehend von gegebener Pose und evtl. vorliegender (Teil-)Karte: Beim Bewegen durch die Umgebung, nur mit Hilfe der Sensorik an Bord, erstell / ergänze eine Karte vorgegebener Modalität (metrisch, topologisch,...)
- ▶ Impliziert, dass der Roboter in der Lage ist, sich zu lokalisieren, um die Karte zu erstellen
- ▶ Zur Lokalisierung benötigt der Roboter aber eigentlich eine Karte...

SLAM (Simultaneous Localization and Mapping), seltener **CML** (Concurrent Localization and Mapping)

Roboterkartierung ist ein Henne-Ei-Problem!

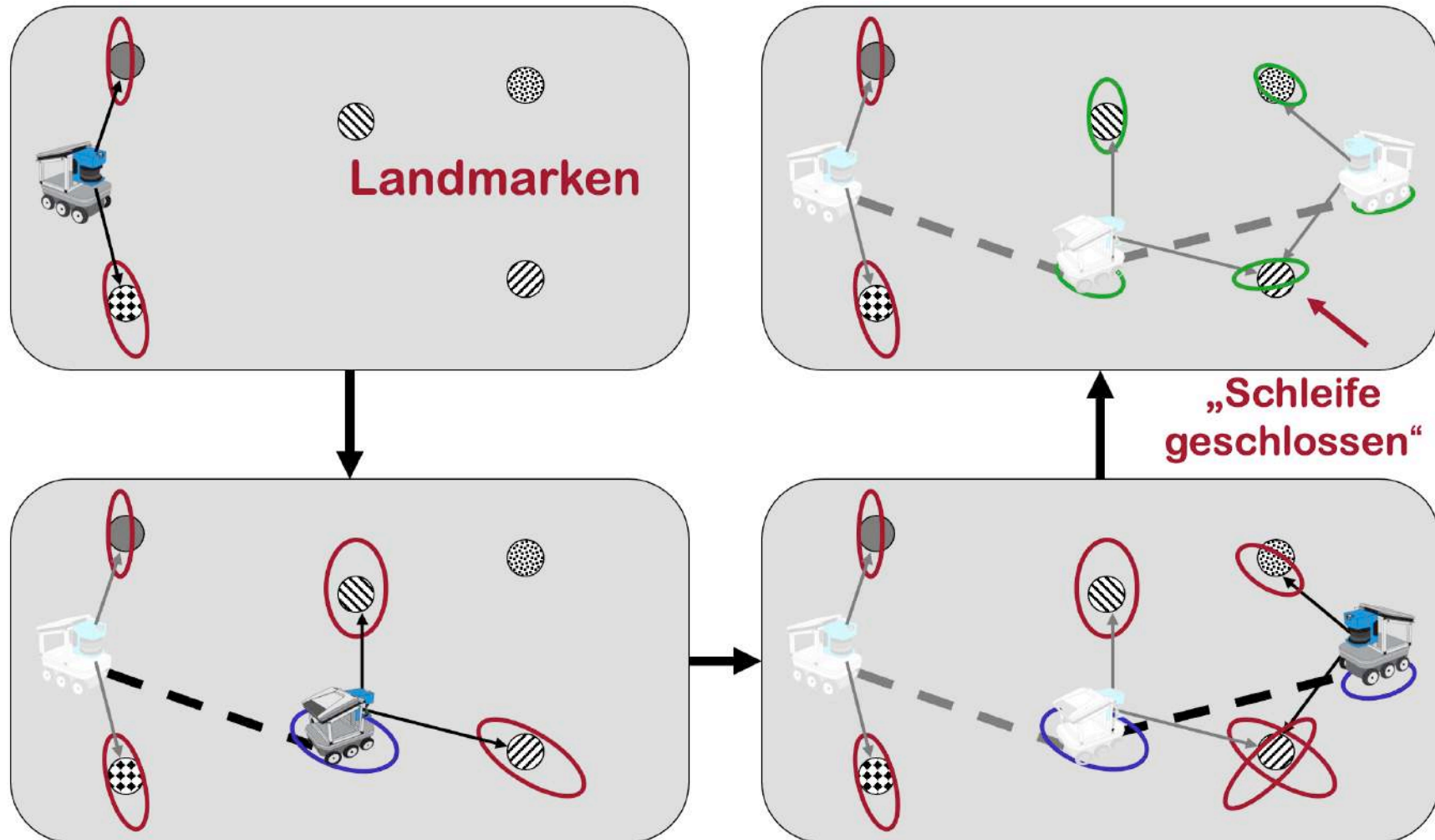
Wandpose → Roboterpose

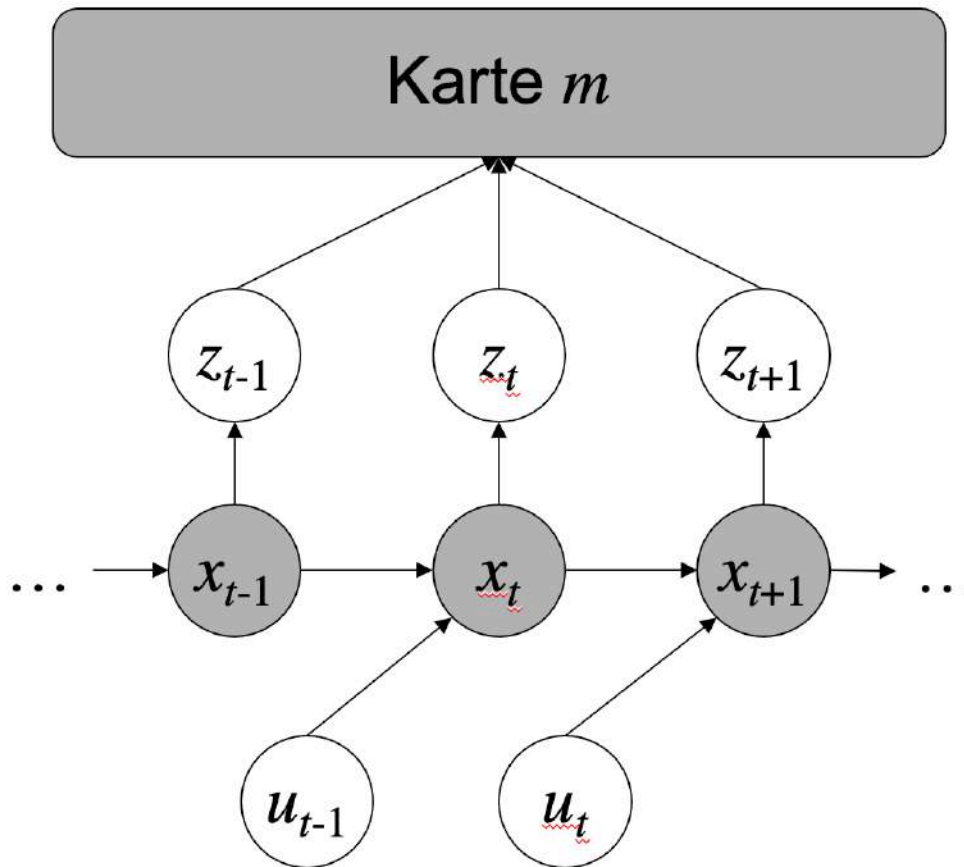


Roboterpose → Wandpose



Intuitive Struktur des Kartierungsproblems





► **Gegeben:**

- Folge von Evidenzen (Beobachtungen z_i von Landmarken / Objekten, Bewegungen u_i), i.a. mit Unsicherheit behaftet

► **Finde:**

- Posen x_i der Beobachtungen und Karte m mit Positionen der Landmarken / Objekte, die x_i und z_i optimal integrieren

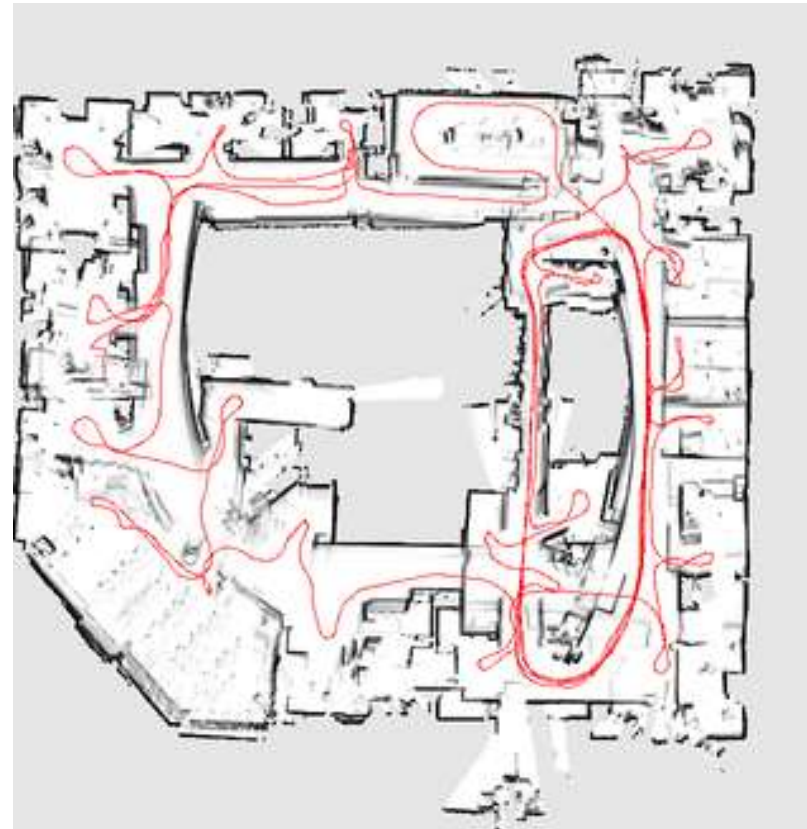
► **Probabilistische Formulierung:**

- Berechne Verteilung $\mathbf{P}(\mathbf{m}, \mathbf{x}_{1:t} \mid z_{1:t}, u_{1:t-1})$



Warum ist das so schwer?

- 2D-Eingabedaten, 3D-Posen!





- **Kartierung bei bekannten Posen:**
„Basisfall“ und fürs Problemverständnis
$$m_t = \operatorname{argmax}_m P(m | z_{1:t}, x_{1:t})$$
- **Inkrementelles SLAM:**
SLAM unter Berücksichtigung nur der aktuellen Pose,
d.h. Berechnung von $\mathbf{P}(m, x_t | z_{1:t}, u_{1:t-1})$
- **Volles SLAM:**
Berechnung von $\mathbf{P}(m, x_{1:t} | z_{1:t}, u_{1:t-1})$



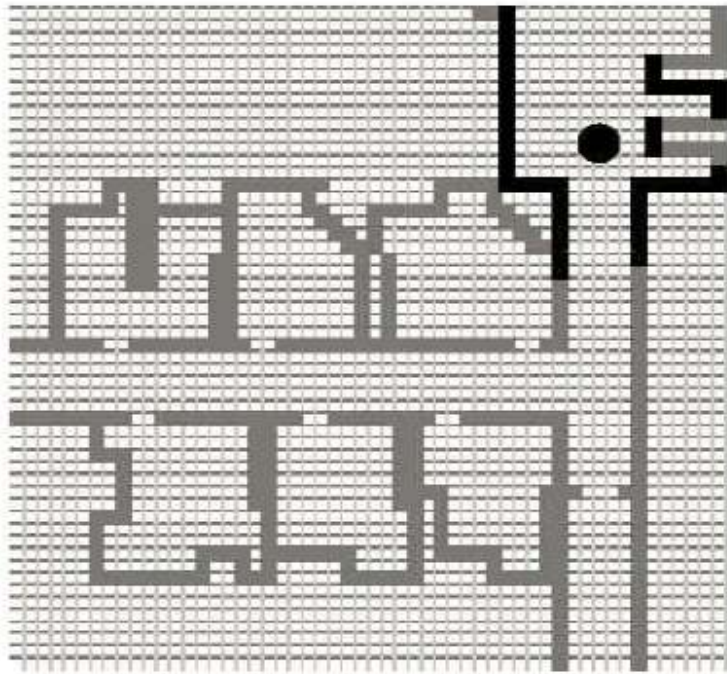
Kartierung bei bekannten Posen

- ▶ ... kommt vor z.B. im Freiland mit GPS (dGPS, RTK-GPS) und Kompass
- ▶ ... ist einfach: $m_t = \operatorname{argmax}_{m_i} P(m_i | z_{1:t}, x_{1:t})$
- ▶ Beispiel: Rasterkarten
 - Initialisiere Felder mit a-priori-Belegtheitswahrscheinlichkeit,
 - Fahre durch die Umgebung,
 - Miss in der Gegend herum,
 - Nach jeder Messung aktualisiere Belegtheitswahrscheinlichkeit der angemessenen Rasterzellen!
 - Aktualisierung geht folgendermaßen ...



Occupancy Mapping (1)

- ▶ Eine Belegtheitskarte besteht aus einer Anzahl an Zellen $m = \{m_1, m_2, \dots, m_n\}$
- ▶ Jede Zelle kann den Wert “frei” oder “belegt” haben (unbekannt: $m = 0.5$)



- ▶ Annahme: Das von einer bekannten Zelle repräsentierte Gebiet ist entweder komplett frei oder belegt



Belegter Raum



Freiraum

- ▶ Jede Zelle ist eine binäre Zufallsvariable, welche die Belegtheitswahrscheinlichkeit repräsentiert



$p(m_i) \rightarrow 1$



$p(m_i) \rightarrow 0$



Occupancy Mapping (2)

- Annahme: Die Zellen (Zufallsvariablen) sind unabhängig von einander



- Die Wahrscheinlichkeitsverteilung der Karte ist definiert durch das Produkt über alle Zellen

$$p(m) = \prod_i p(m_i)$$

Binärer Bayes-Filter für einen
statischen Zustand

Binäre Zufallsvariable



Occupancy Grid Mapping - Algorithmus

Eingabe : Sequenz von Messwerten z_t und korrekten Posen x_t .

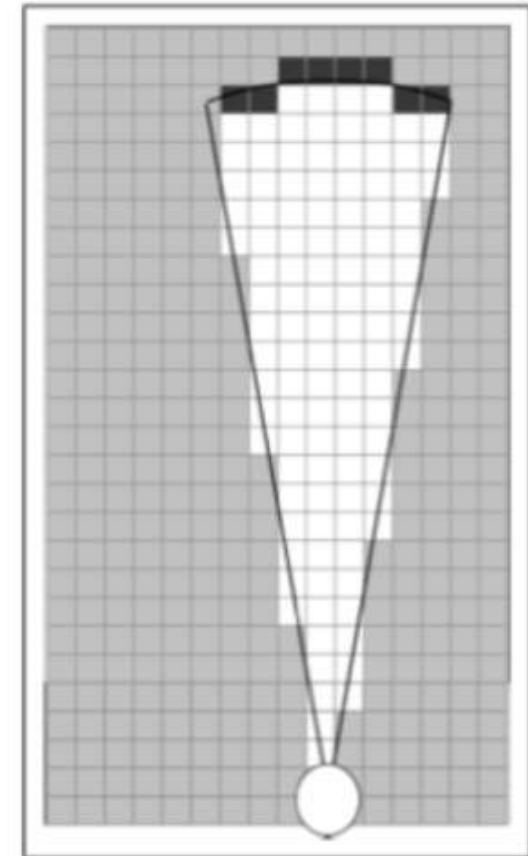
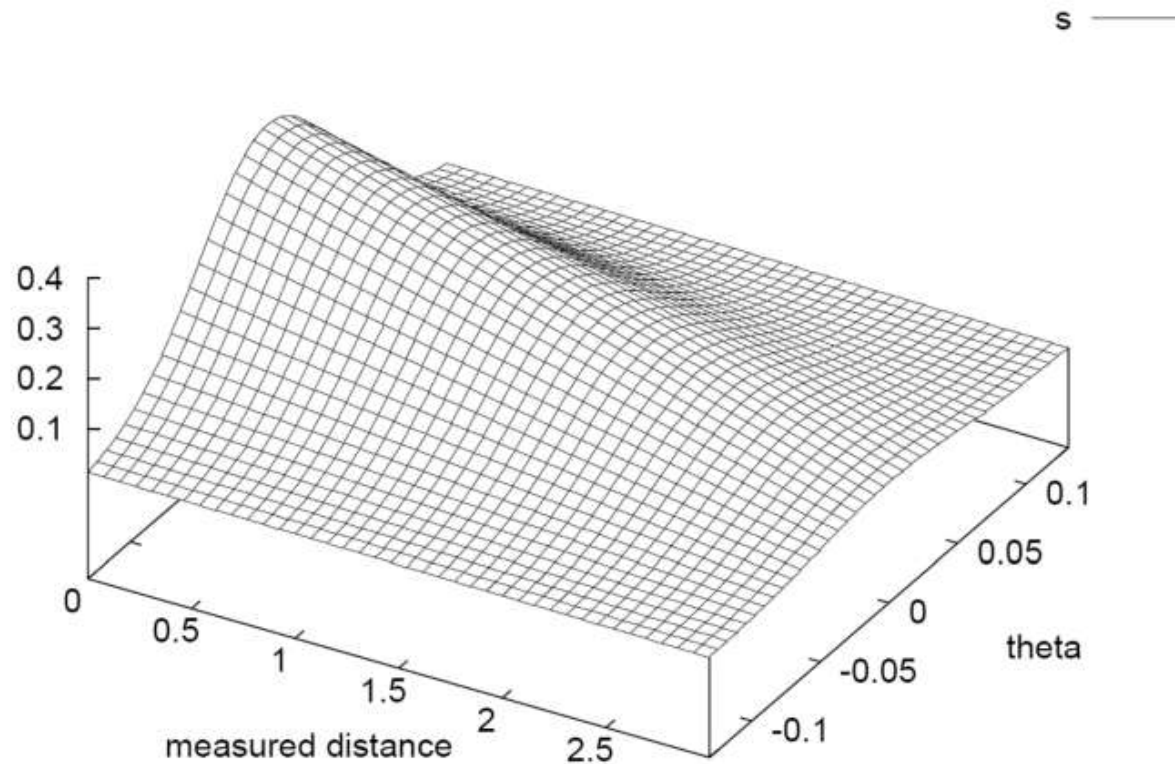
Ausgabe: Belegtheitswahrscheinlichkeit $Bel(m_{x,z})$ für jede Rasterzelle (x, z) .

```
1: for all  $m_{x,z} \in m$  do
2:    $Bel(m_{x,z}) = P_{\text{initial}}(m_{x,z})$ 
3: end for
4: for all  $t$  do
5:    $Bel(m_{x,z,t}) = 0$                                      // Roboter befindet sich in der Pose  $(x, z, \theta)$ .
6:                                     // Diese Position ist daher mit Sicherheit Freiraum.
7: end for
8: for all  $m_{x,z,t} \in m$  do
9:   if Rasterzelle  $(x, z)$  ist in Sensorreichweite then
10:     $Bel(m_{x,z,t}) = 1 - \left( 1 + \frac{P(m_{x,z,t} | z_t, u_t)}{1 - P(m_{x,z,t} | z_t, u_t)} \frac{1 - P(m_{x,z,t})}{P(m_{x,z,t})} \frac{Bel(m_{x,z,t-1})}{1 - Bel(m_{x,z,t-1})} \right)^{-1}$ 
11:   end if
12: end for
13: return  $Bel(m_{x,z})$ 
```



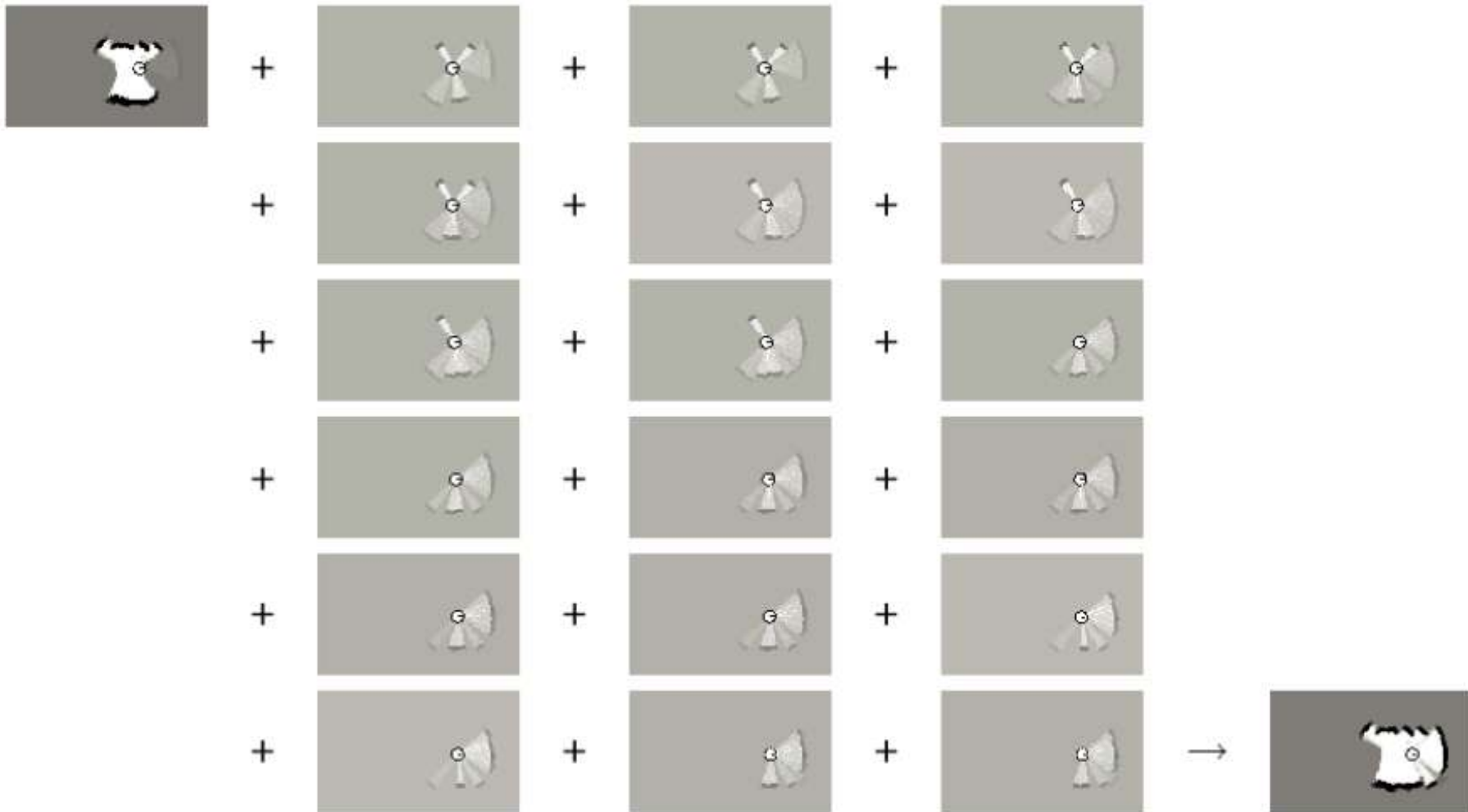
Beispiel Mapupdate

- “Intensität” des Kartenupdates:





Beispiel - Mapping mit Ultraschall





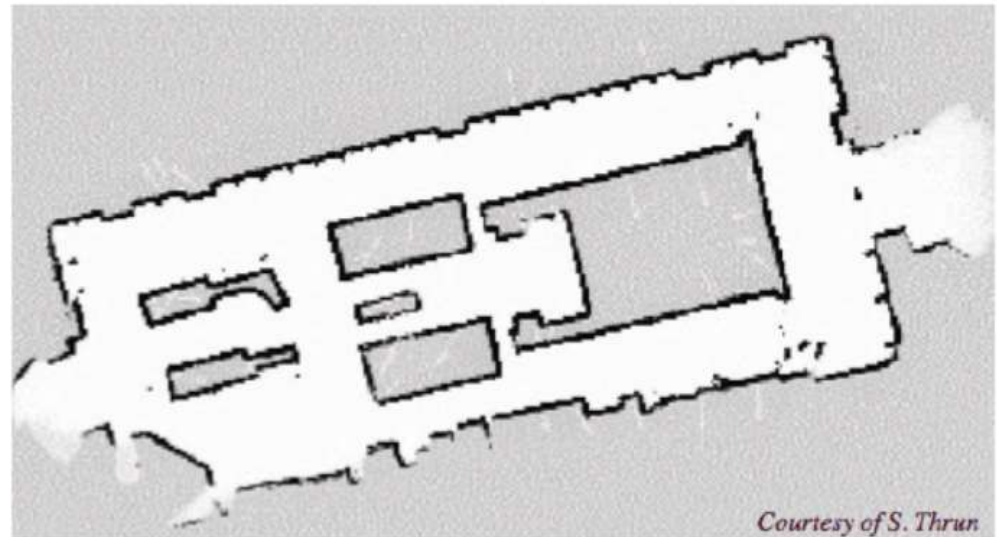
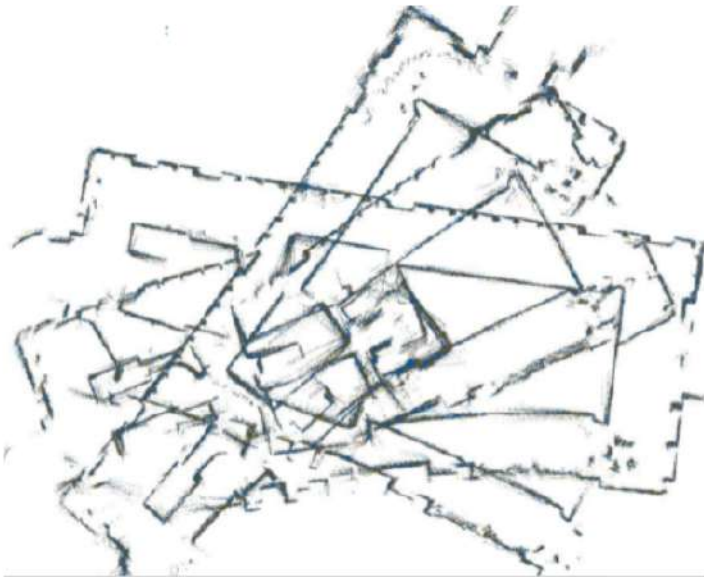
Wenn wir wüssten, wo wir sind

- ▶ ... könnten wir mit dem vorgestellten Verfahren eine Rasterkarte mit Hilfe des inversen Sensormodells berechnen:
 - Initialisiere alle Zellen mit einer A-Prio-Wahrscheinlichkeit für Belegtheit
 - Bewege Dich durch die Umgebung
 - Messe die Abstände und Update die Karte



Wir wissen es aber nicht!

- ▶ Messfehler sind z.T. unbeschränkt und statistisch nicht unabhängig (Odometrie!)
- ▶ Kartierung ist ein sehr hochdimensionales Problem (z.B. n Laserscans à 181 2D-Punkte)
- ▶ Assoziation von Sensordaten (mehrere Messdaten bezeichnen selben Punkt / selbes Umgebungsmerkmal) schwer zu ermitteln ohne Karte
- ▶ Schleifenschluss ist *die* Herausforderung beim Kartieren!



Courtesy of S. Thrun



Grundidee: $P(m, x_t | z_{1:t}, u_{1:t})$

- ▶ Tracke die Schätzung der aktuellen Pose (und nur die), wobei folgende Abhängigkeiten bestehen:
 - Neue Karteneinträge hängen von der aktuellen Poseschätzung ab („Wo ich eine gerade gesehene Belegtheit / Landmarke in die Karte eintrage, hängt von der Pose ab“)
 - Die Poseschätzung hängt (unter anderem) ab von Lokalisierung an aktuell wahrgenommenen Landmarken / Belegtheiten, die bereits in der Karte sind
 - ⇒ Wiederverwendung von Lokalisierungs-Methoden
 - ⇒ hier: EKF und Partikelfilter/MCL



EKF-SLAM: Die Idee

- ▶ Geh vor wie bei EKF-Lokalisierung mit bekannter Karte (vorheriger Abschnitt), aber:
- ▶ Erweitere Zustandsraum von Pose ($x = (t_x, t_z, \theta)$) auf Pose plus Koordinaten (hier: x, z) aller Landmarken
- ▶ Setze voraus, Landmarken sind eindeutig und unterscheidbar (kein „aliasing“)
- ▶ Falls Zahl der Landmarken unbekannt, arbeite also mit Zustandsraum variabler Größe (wächst mit Zahl wahrgenommener Landmarken)
- ▶ Wende EKF an wie gehabt, wobei alle zustandsabhängigen Matrizen „mitwachsen“ können



Prinzip des EKF-SLAM-Algorithmus

- ▶ Berechne a-priori-Roboterpose-Wert und -Kovarianz aus Bewegungsmodell (Landmarken zunächst unverändert) $\Rightarrow \underline{\mu}_t, \underline{\Sigma}_t$
- ▶ Für erstmalig gesehene Landmarken füge Zustandsraumdimensionen hinzu. Initialisiere ihre (x, z) -Werte über aktuelle Wahrnehmungen (umgerechnet in globale Koordinaten)
- ▶ Mittels aller sichtbarer Landmarken aktualisiere $\underline{\mu}_t, \underline{\Sigma}_t$ (berechne Kalman-Gewinnmatrix etc. dafür wie üblich)
- ▶ Das Ergebnis am Ende ist $\underline{\mu}_t, \underline{\Sigma}_t$ a posteriori: Neuer Wert + Kovarianzen für Roboterpose und Landmarkenpositionen

Bemerkungen:

- Vollständige Formulierung des Algorithmus: Thrun / Fox / Burgard Kap.10.2
- Dort mit fest vorgegebener (Maximal-)Zahl von Landmarken und mit iterativer Berechnung von $\underline{\mu}_t, \underline{\Sigma}_t$ über die aktuell gesehenen Landmarken



EKF-SLAM mit bekannten Landmarken (1)

- Zustandsrepräsentation:

$$\mathbf{x}_k = (x_r, y_r, x_{l1}, y_{l1}, x_{l2}, y_{l2}, \dots, x_{ln}, y_{ln})^T$$

- Bewegungsmodell:

$$\mathbf{u}_k = (v_x \quad v_y)^T$$

- Transitionsmodell:

$$\mathbf{x}_k = \mathbf{x}_{k-1} + \begin{pmatrix} \Delta t & 0 \\ 0 & \Delta t \\ 0 & 0 \\ 0 & 0 \\ \vdots & \vdots \\ 0 & 0 \end{pmatrix} \mathbf{u}_k$$

Landmarken
bewegen sich nicht!



EKF-SLAM mit bekannten Landmarken (2)

- ▶ Landmarken sind eindeutig unterscheidbar
- ▶ Im allgemeinen sind nicht alle Landmarken immer sichtbar
- ▶ Jede Landmarke wird relativ zur Roboterposition gemessen

$$\mathbf{z}_k = \begin{pmatrix} x_{t1} - x_r \\ y_{t1} - y_r \\ \vdots \\ x_{tn} - x_r \\ y_{tn} - y_r \end{pmatrix}$$

- ▶ Sensormodell:

$$\mathbf{z}_k = \begin{pmatrix} H_1 \\ \vdots \\ H_n \end{pmatrix} \mathbf{x}_t$$

$$H_i = \begin{pmatrix} -1 & 0 & 0 & \dots & 0 & 1 & 0 & 0 & \dots & 0 \\ 0 & -1 & 0 & \dots & 0 & 0 & 1 & 0 & \dots & 0 \end{pmatrix}$$

$\underbrace{\hspace{1.5cm}}_{(2i+1)\text{-te Spalte}}$



EKF-SLAM mit bekannten Landmarken (3)

- ▶ Das System ist jetzt vollständig beschrieben
- ▶ Wir können also den Kalman-Filter anwenden

KalmanFilter($\hat{\mathbf{x}}_k, \Sigma_k, \mathbf{u}_k, \mathbf{z}_{k+1}$):

1. Vorhersage:
2. $\hat{\mathbf{x}}_{k+1}^- = \mathbf{A}\hat{\mathbf{x}}_k + \mathbf{B}\mathbf{u}_k$
3. $\Sigma_{k+1}^- = \mathbf{A}\Sigma_k\mathbf{A}^T + \mathbf{B}\Sigma_u\mathbf{B}^T$
4. Korrektur:
5. $\hat{\mathbf{z}}_{k+1} = \mathbf{C}\hat{\mathbf{x}}_{k+1}^-$
6. $\mathbf{K} = \Sigma_{k+1}^- \mathbf{C}^T (\mathbf{C}\Sigma_{k+1}^- \mathbf{C}^T + \Sigma_z)^{-1}$
7. $\hat{\mathbf{x}}_{k+1} = \hat{\mathbf{x}}_{k+1}^- + \mathbf{K}(\mathbf{z}_{k+1} - \hat{\mathbf{z}}_{k+1})$
8. $\Sigma_{k+1} = (\mathbf{I} - \mathbf{K}\mathbf{C})\Sigma_{k+1}^-$
9. **return** ($\hat{\mathbf{x}}_{k+1}, \Sigma_{k+1}$)



► Initialisierung

$$\hat{\mathbf{x}}_0 = \underbrace{(0 \ 0 \ 0 \ \dots \ 0)}_{2*n+2}^T$$

$$\Sigma_0 = \underbrace{\begin{pmatrix} 0 & 0 & 0 & \dots & 0 \\ 0 & 0 & 0 & \dots & 0 \\ 0 & 0 & \infty & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & \infty \end{pmatrix}^T}_{2*n+2}$$

Die Startposition des Roboters definiert das Koordinatensystem der Karte

An der Startposition haben wir keine Unsicherheiten

Alle Landmarken sind unbekannt



Kovarianzmatrix für EKF-SLAM

- ▶ Seien n Landmarken in Karte eingetragen (je mit x - und z -Wert)
- ▶ Zustandsraumgröße aktuell also $3 + 2n$

Unsicherheit der
Roboterpose

Kovarianzen sind Teil der Karte!

$$\Sigma_t = \begin{pmatrix} \sigma_x^2 & \sigma_{x,z} & \sigma_{x,\theta} & \sigma_{x,l_1,x} & \sigma_{x,l_1,z} & \cdots & \sigma_{x,l_n,x} & \sigma_{x,l_n,z} \\ \sigma_{x,z} & \sigma_z^2 & \sigma_{z,\theta} & \sigma_{z,l_1,x} & \sigma_{z,l_1,z} & \cdots & \sigma_{z,l_n,x} & \sigma_{z,l_n,z} \\ \sigma_{x,\theta} & \sigma_{x,\theta} & \sigma_\theta^2 & \sigma_{\theta,l_1,x} & \sigma_{\theta,l_1,z} & \cdots & \sigma_{\theta,l_n,x} & \sigma_{\theta,l_n,z} \\ \sigma_{l_1,x,x} & \sigma_{l_1,x,z} & \sigma_{l_1,x,\theta} & \sigma_{l_1,x}^2 & \sigma_{l_1,x,l_1,z} & \cdots & \sigma_{l_1,x,l_n,x} & \sigma_{l_1,x,l_n,z} \\ \sigma_{l_1,z,x} & \sigma_{l_1,z,z} & \sigma_{l_1,z,\theta} & \sigma_{l_1,z,l_1,x} & \sigma_{l_1,z}^2 & \cdots & \sigma_{l_1,z,l_n,x} & \sigma_{l_1,z,l_n,z} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ \sigma_{l_n,x,x} & \sigma_{l_n,x,z} & \sigma_{l_n,x,\theta} & \sigma_{l_n,x,l_1,x} & \sigma_{l_n,x,l_1,z} & \cdots & \sigma_{l_n,x}^2 & \sigma_{l_n,x,l_n,z} \\ \sigma_{l_n,z,x} & \sigma_{l_n,z,z} & \sigma_{l_n,z,\theta} & \sigma_{l_n,z,l_1,x} & \sigma_{l_n,z,l_1,z} & \cdots & \sigma_{l_n,z,l_n,x} & \sigma_{l_n,z}^2 \end{pmatrix}$$

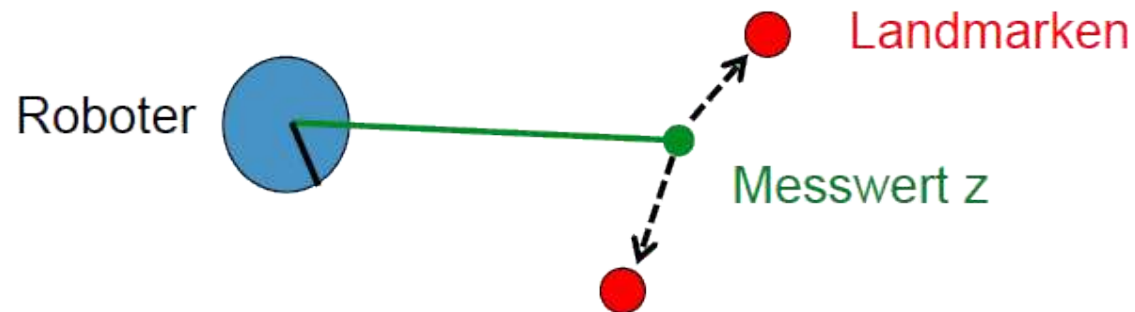
Korrelationen Roboterposen /
Landmarkenpositionen

Unsicherheit der Landmarkenpositionen

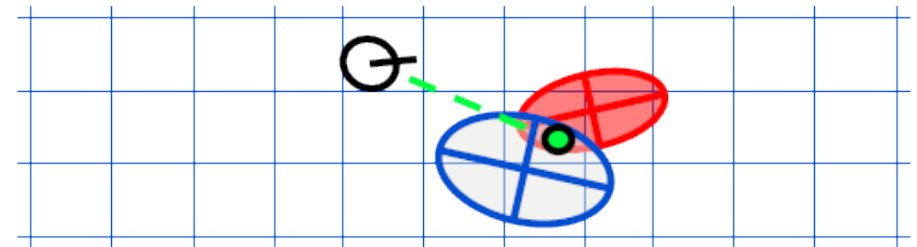


EKF-SLAM mit unbekannter Assoziation

- Finde für jede Messung die wahrscheinlichste Landmarke

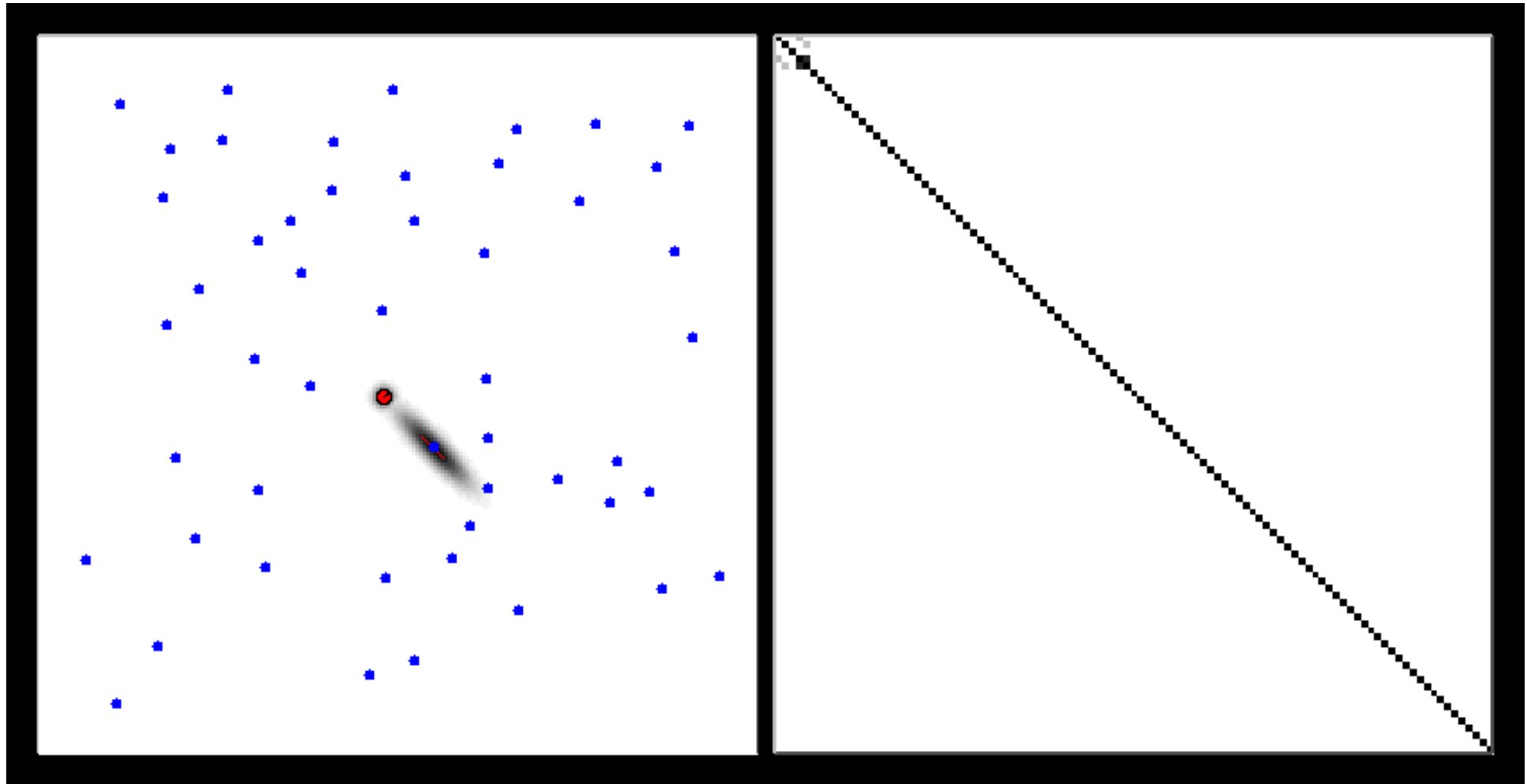


- Auswahl der Landmarke mit der geringsten Unsicherheit
- Oft ist das die nächstgelegenste



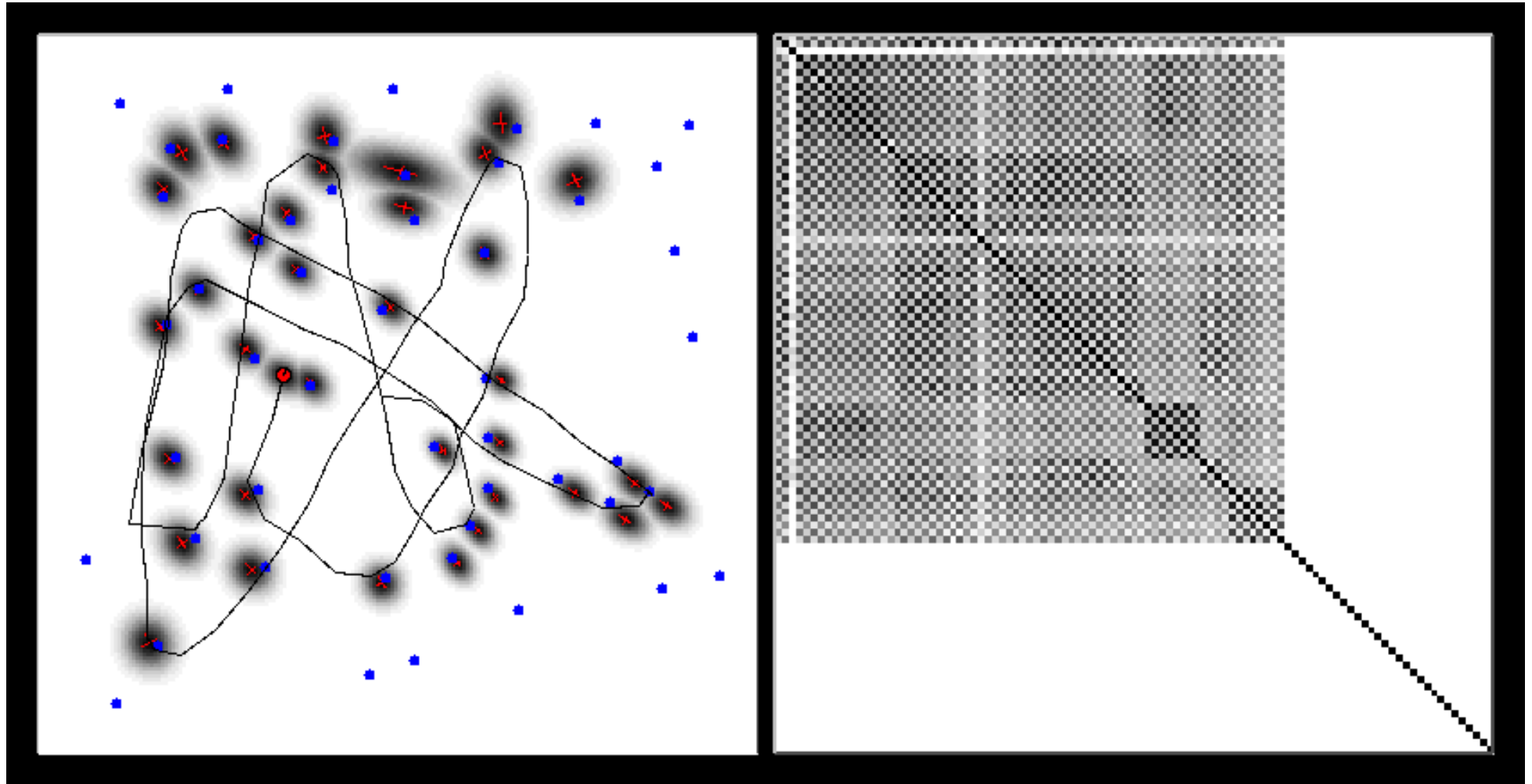


Beispiel EKF-SLAM (1)



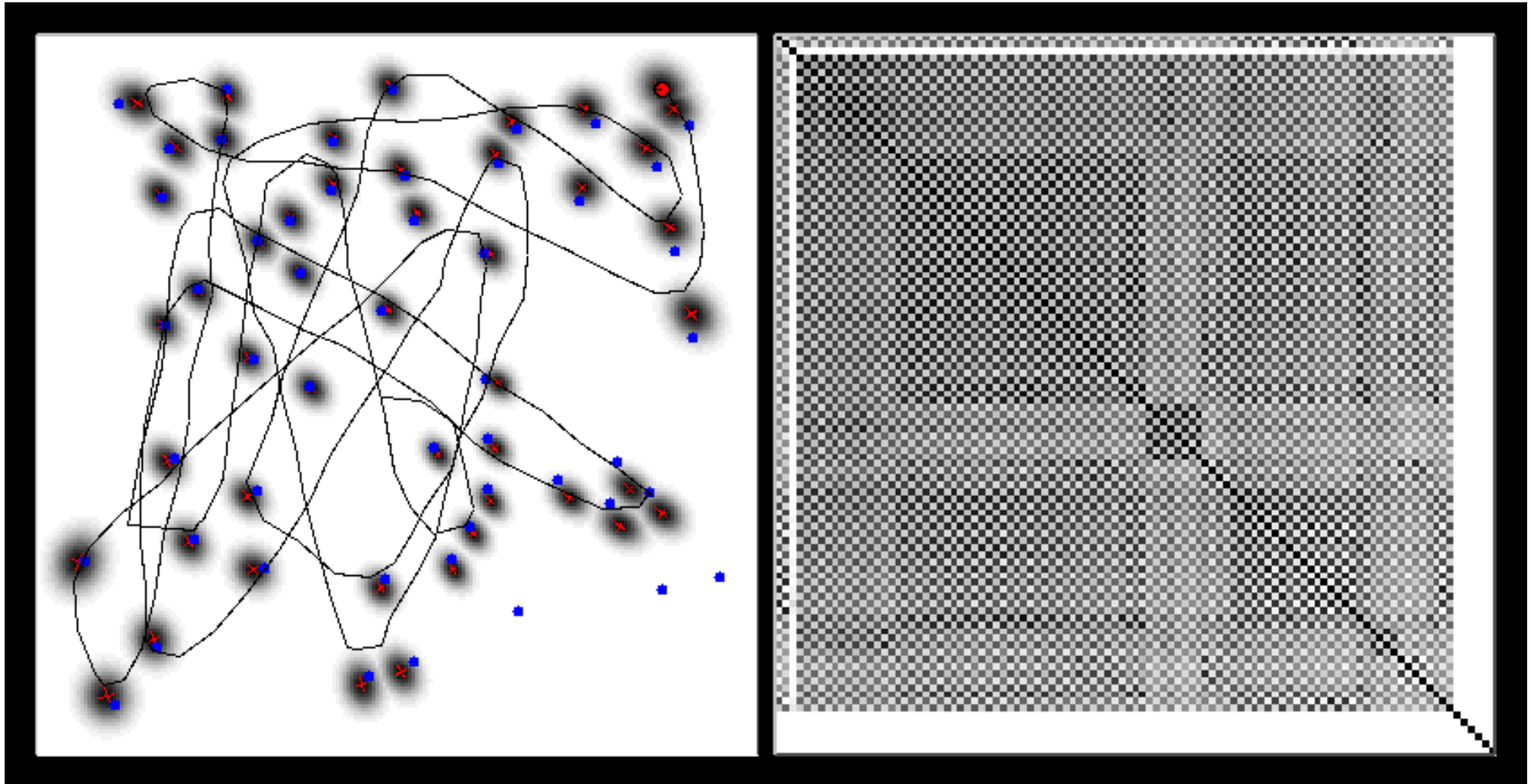


Beispiel EKF-SLAM (2)





Beispiel EKF-SLAM (3)





Satz:

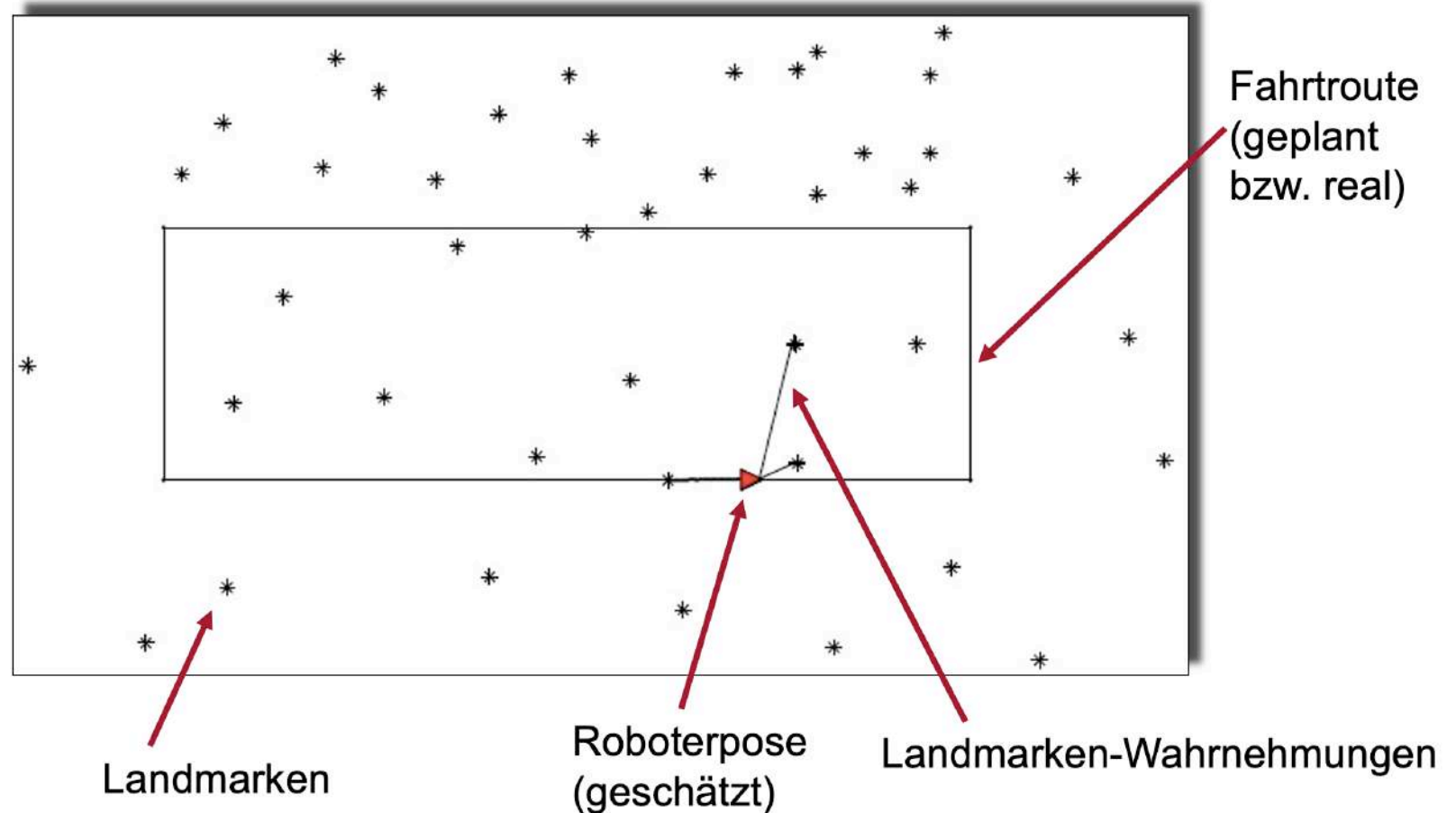
Im Limes der Roboterfahrtlänge sind die Schätzungen der Landmarken vollständig korreliert.

Dissanayake et al.

- ▶ Landmarkenpositionen unabhängig voneinander
- ▶ Roboterpose bleibt dabei potenziell mit Unsicherheit behaftet!
- ▶ **Nachteile von EKF-SLAM:**
 - Das Verfahren kann nicht die Information verarbeiten, dass an der aktuellen Pose nichts gesehen wird!
 - Kovarianzmatrix wächst mit $\mathcal{O}(n^2)$ in der Anzahl der Landmarken
 - Bei großen Nicht-Linearitäten kann das Verfahren divergieren

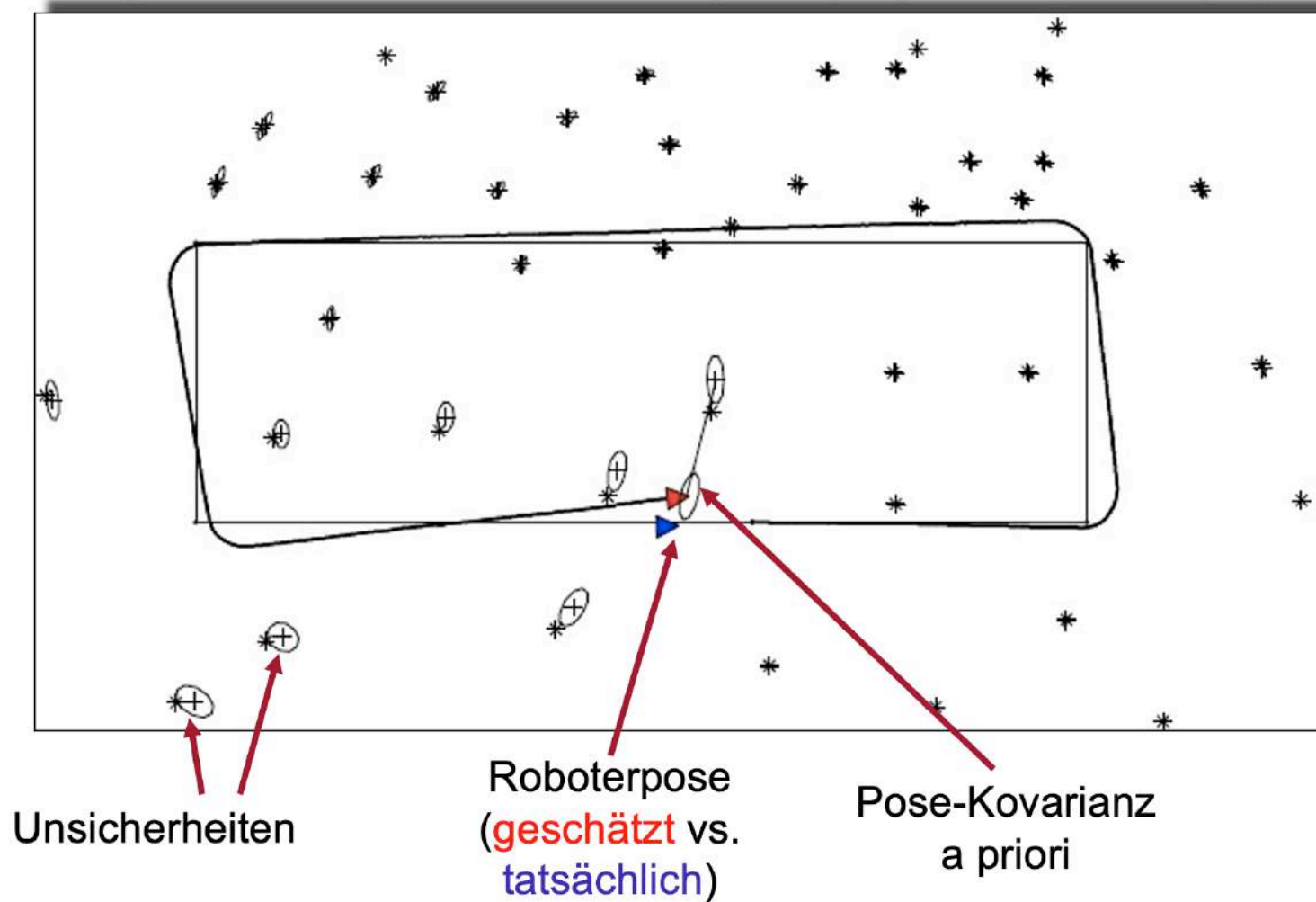


Synthetisches Beispiel (1)



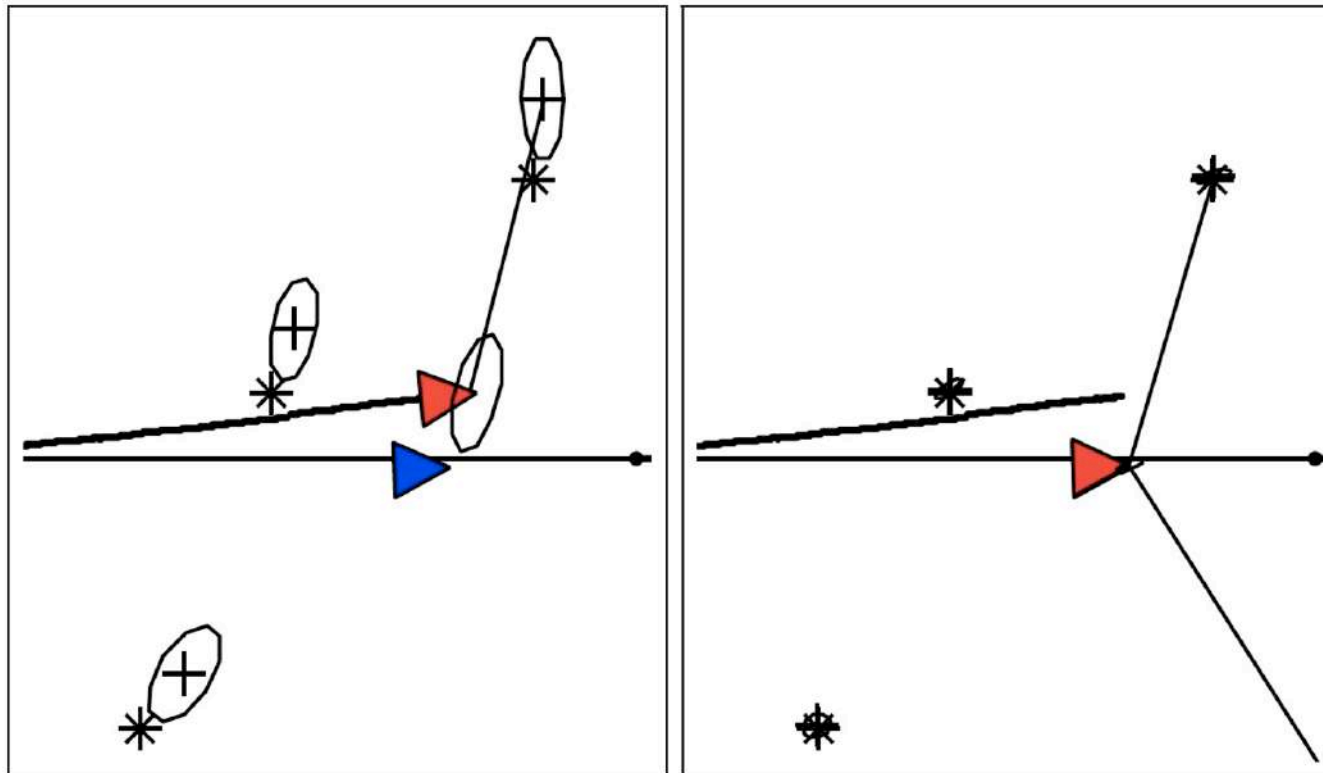


Synthetisches Beispiel (2)





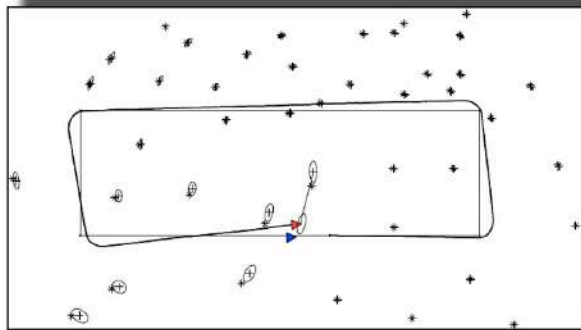
Synthetisches Beispiel (3)



A-priori- vs. a posteriori-Poseschätzungen im Fall des Schleifenschlusses



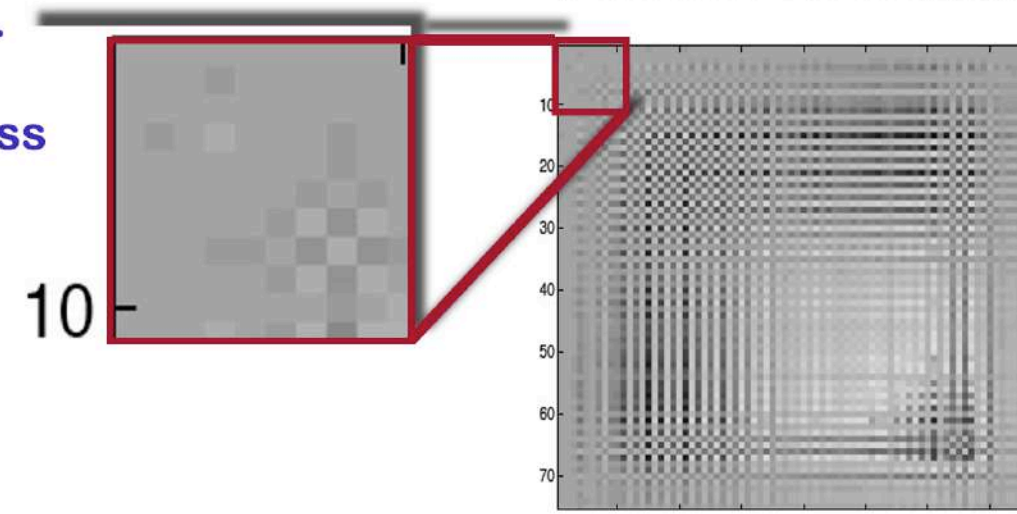
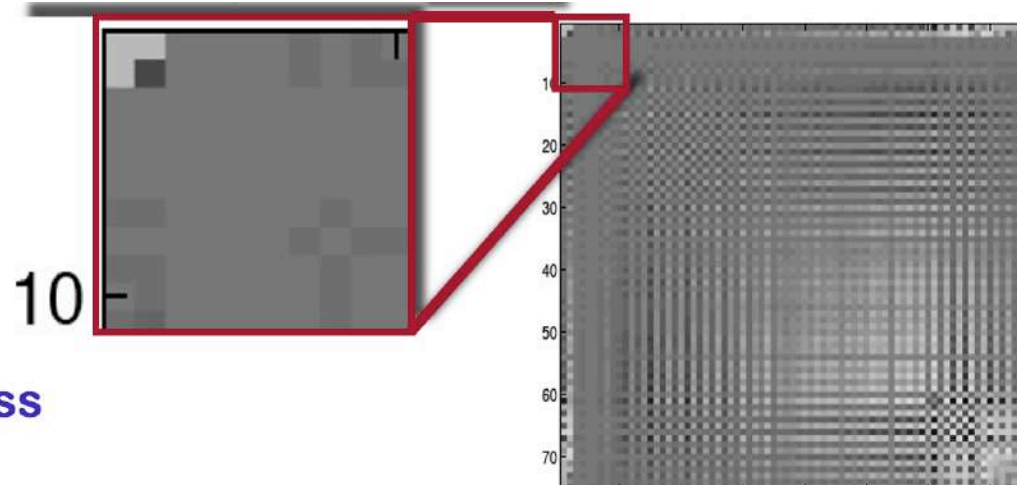
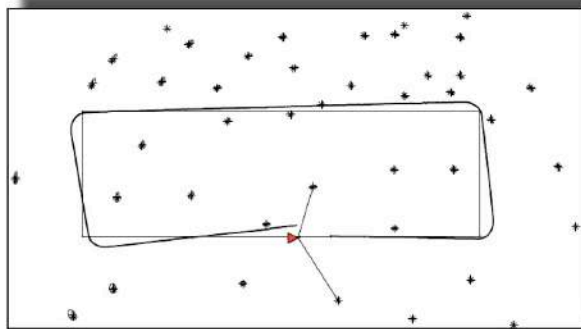
Entwicklung von Σ_t im Beispiel (2)



... vor Schleifenschluss

Alle Landmarken gesehen ...

... nach Schleifenschluss





- ▶ Jedes Partikel hält aktuelle Pose (alternativ: kompletten Pfad mit aktueller Pose als letzter) und die Schätzung der Positionen der Landmarken (feste Landmarkenzahl n)
- ▶ Errechne neue Pose durch Sampling vom Bewegungsmodell
- ▶ Problem: Um Wahrscheinlichkeitsverteilung im $(3 + 2n)$ -dimensionalen Zustandsraum durch Partikel gut zu approximieren, bräuchte man riesig viele Partikel
- ▶ Lösung: aktualisiere Landmarkenpositionen durch n „Mini“-EKF für Positionen der einzelnen, aktuell sichtbaren Landmarken

Literatur:

Montemerlot, Thrun & al.: *FastSLAM: A Factored Solution to the Simultaneous Localization and Mapping Problem*. Proc. AAAI-2002

Buch Kap. 6.3.2

Thrun / Burgard / Fox, Kapitel 13



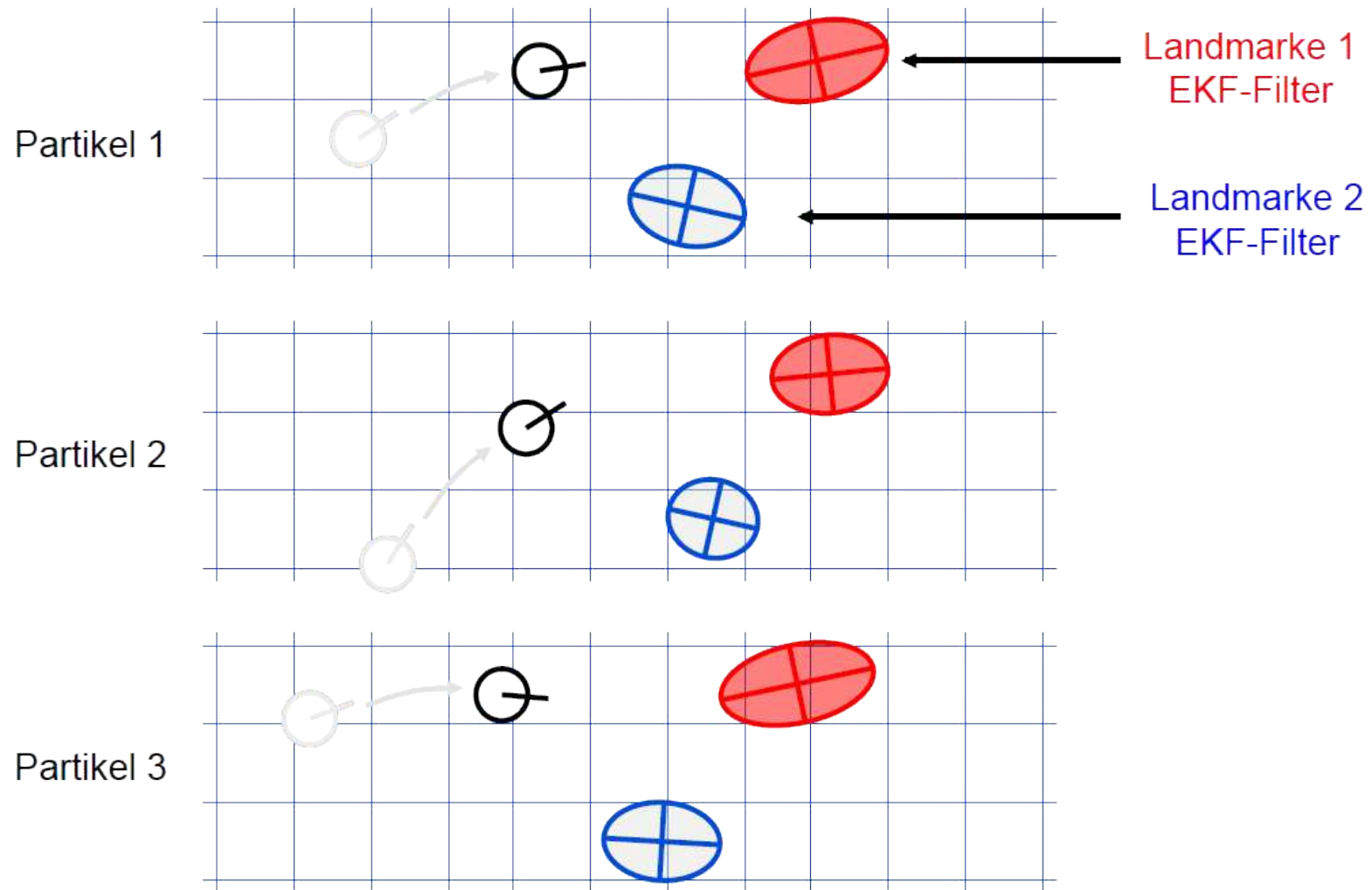
Implementierung über Partikelfilter

- ▶ N Partikel, jedes mit n „kleinen EKF“ für n Landmarken
- ▶ Beschränkung auf jeweils aktuelle Posen: Inkrementelles SLAM

	Pose (volles SLAM: Pfad)	Position <u>Landm. 1</u>	Position <u>Landm. 2</u>	Position <u>Landm. n</u>
Partikel <u>$k=1$</u>	$s^{[1]} = (x, z, \theta)^{T[1]}$	$\mu_1^{[1]}, \Sigma_1^{[1]}$	$\mu_2^{[1]}, \Sigma_2^{[1]}$	$\dots \mu_n^{[1]}, \Sigma_n^{[1]}$
Partikel <u>$k=2$</u>	$s^{[2]} = (x, z, \theta)^{T[2]}$	$\mu_1^{[2]}, \Sigma_1^{[2]}$	$\mu_2^{[2]}, \Sigma_2^{[2]}$	$\dots \mu_n^{[2]}, \Sigma_n^{[2]}$
	$\vdots$			
Partikel <u>$k=N$</u>	$s^{[N]} = (x, z, \theta)^{T[N]}$	$\mu_1^{[N]}, \Sigma_1^{[N]}$	$\mu_2^{[N]}, \Sigma_2^{[N]}$	$\dots \mu_n^{[N]}, \Sigma_n^{[N]}$



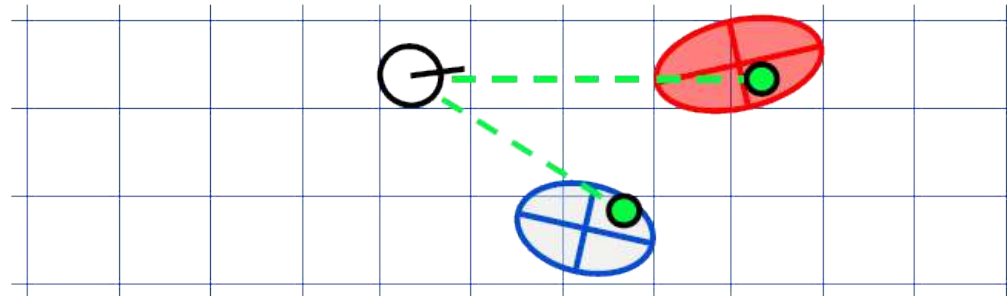
Struktur der N Partikel





Integration der Messungen (1)

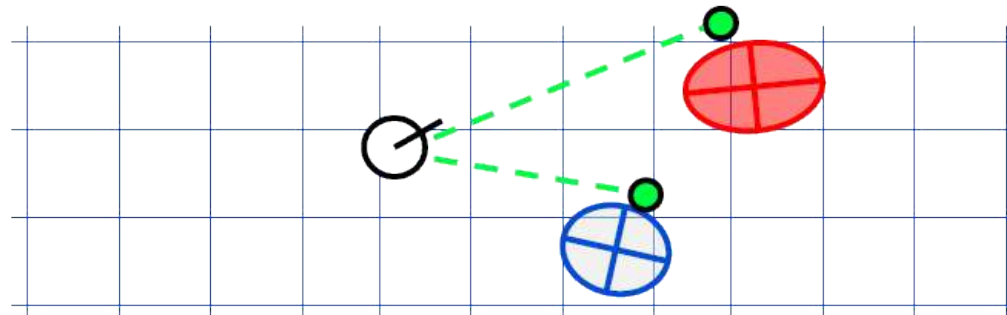
Partikel 1



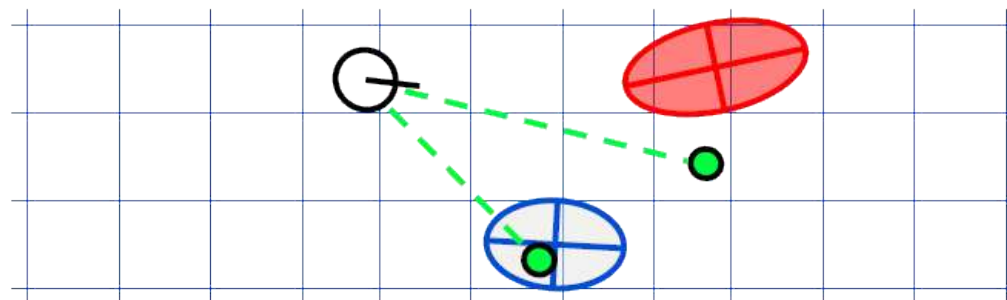
Landmarke 1
EKF-Filter

Landmarke 2
EKF-Filter

Partikel 2



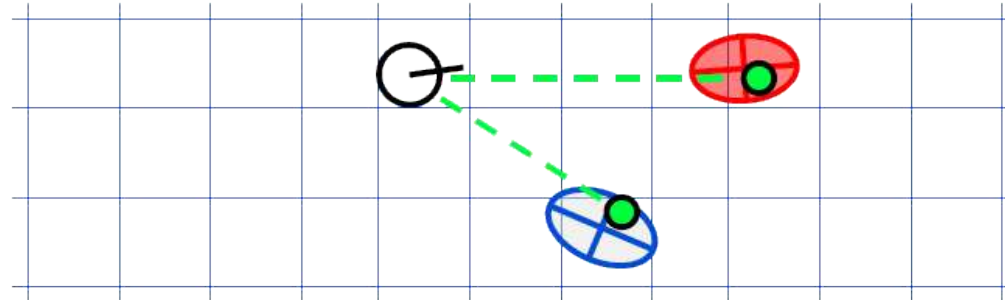
Partikel 3





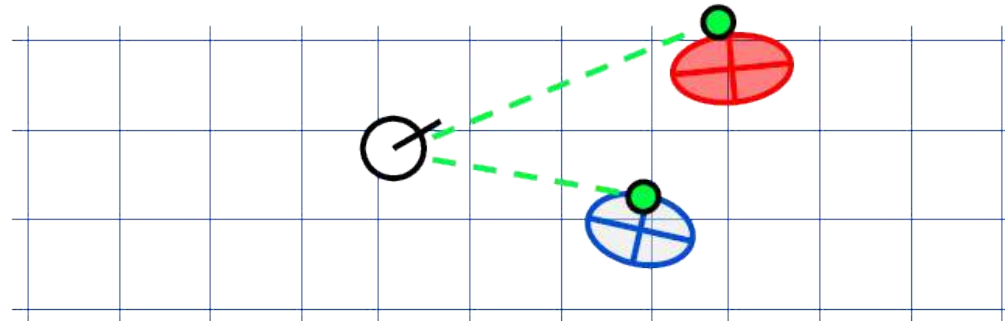
Integration der Messungen (2)

Partikel 1



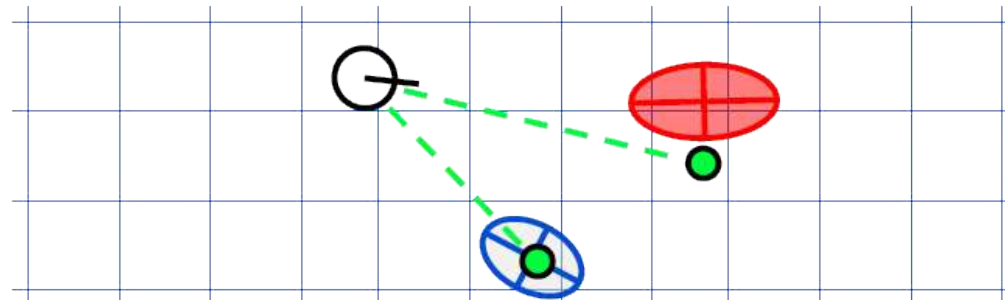
Gewicht = 0.8

Partikel 2



Gewicht = 0.4

Partikel 3

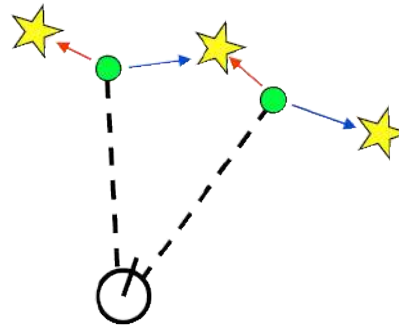


Gewicht = 0.1



Zuordnung der Landmarken

- Zu welchen Landmarken passen die Messungen



- Die Zuordnung erfolgt unabhängig für jedes der N Partikel





Voraussetzung:

Alle Landmarken werden erkannt und nicht verwechselt!

Tu N -mal (Anzahl Partikel) zur Zeit t :

- Ziehe gemäß Gewichtsverteilung Partikel k mit Pose $s^{[k]}$
- Ziehe für k eine neue Pose $s_t^{[k]} \propto P(s_t | s_{t-1}^{[k]}, u_{t-1})$
- Für jede Beobachtung z_t^i bestimme korrespondierende Landmarke(n) j ; aktualisiere $\mu_{j,t}^{[k]}$ und $\Sigma_{j,t}^{[k]}$
- Berechne Partikelgewicht $W^{[k]}$ des neuen Partikels („Wie gut passen Landmarken-Positionen zu Pose/Pfad?“)

Ziehe aus den N aktuellen Partikeln N frische Partikel
(analog Algorithmus MCL, Folie 225)



Grid-based FastSLAM (1)

Rasterkarte

$$p(x_{1:k}, m \mid z_{1:k}, u_{1:k}) = p(x_{1:k} \mid z_{1:k}, u_{1:k}) \cdot p(m \mid x_{1:k}, z_{1:k})$$

Analog MCL

Kartierung mit bekannten Posen



Grid-based FastSLAM (2)

Algorithmus FastSLAM_OccupancyGrids(χ_{k-1} , u_k , z_k):

for $p = 1$ to N do

Integration des Steuerbefehls:

$x_k^{[p]} = \text{sampleMotionModel}(u_k, x_{k-1}^{[p]});$

Gewicht für jeden Partikel brechnen:

$w_k^{[p]} = \text{measurementModel}(z_k, x_k^{[p]}, m_{k-1}^{[p]});$

Integration des Sensordaten:

$m_k^{[p]} = \text{updateOccupancyGrid}(m_{k-1}^{[p]}, x_k^{[p]}, z_k);$

endfor

Resampling:

$\chi_k = \emptyset;$

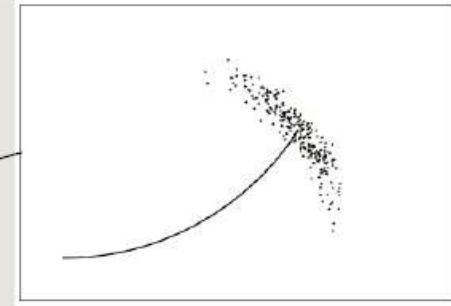
for $i = 1$ to N do

ziehe p zufällig mit Wahrscheinlichkeit $w_k^{[p]}$;

$\chi_k = \chi_k \cup \{ \langle x_{1:k}^{[p]}, m_k^{[p]} \rangle \};$

endfor

return $\chi_k;$

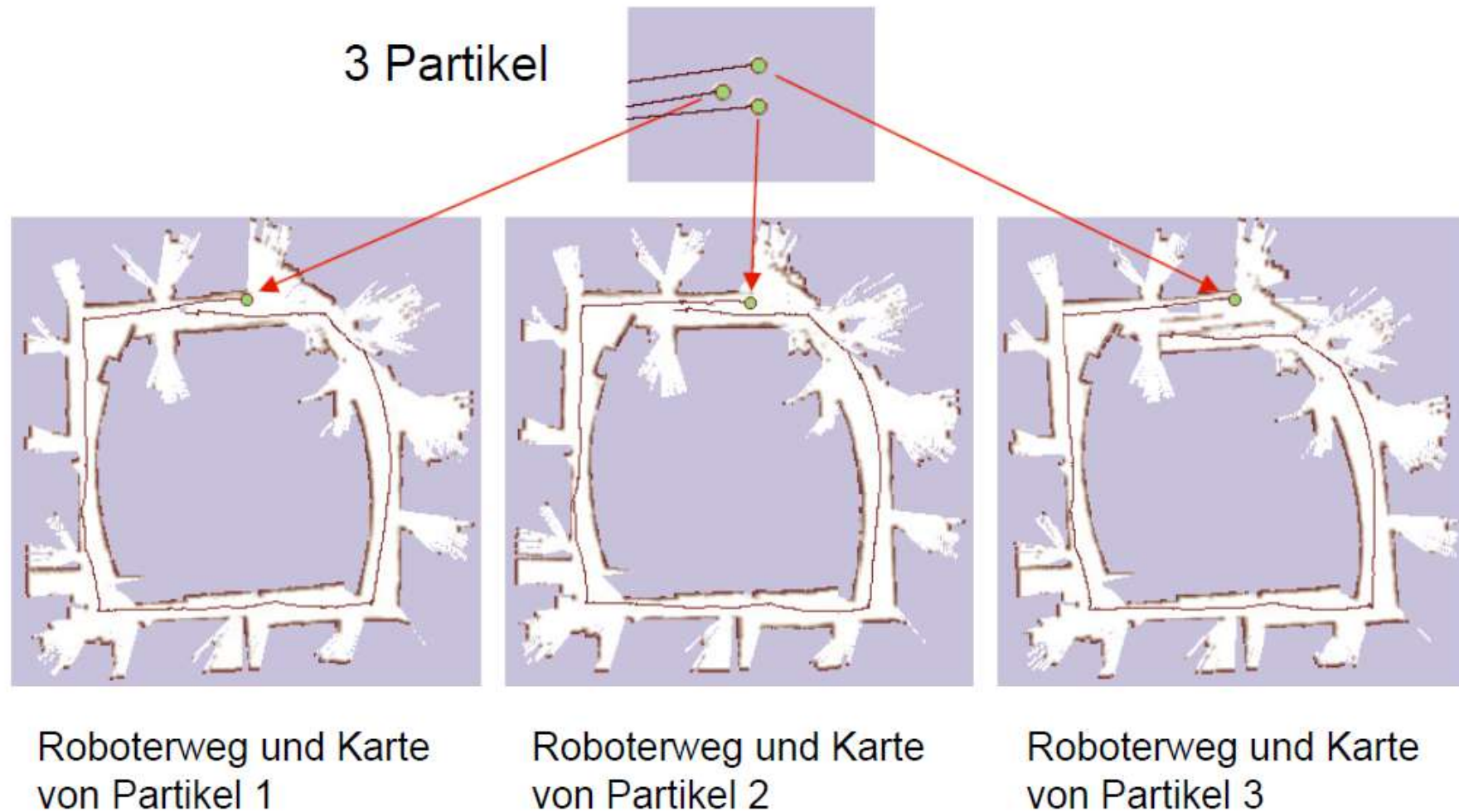


wie beim MCL-Algorithmus
aus Kap. 7

wie occupancyGridMapping
aus Kap 8



Grid-based FastSLAM (3)





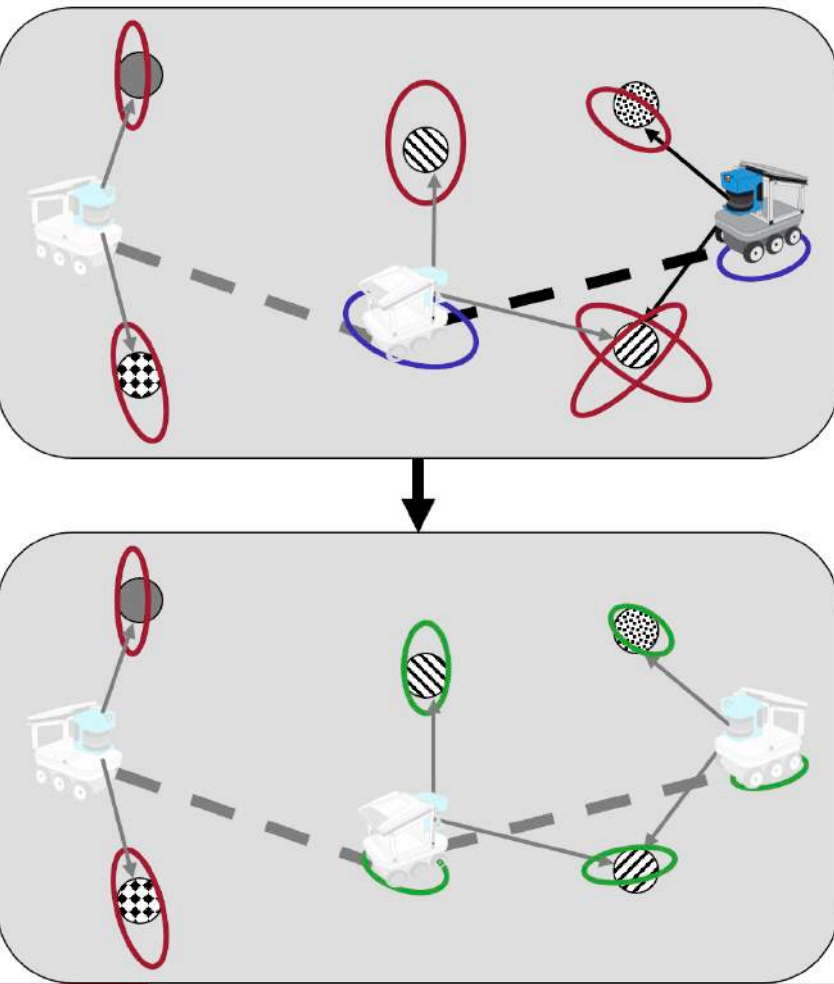
Video FastSLAM



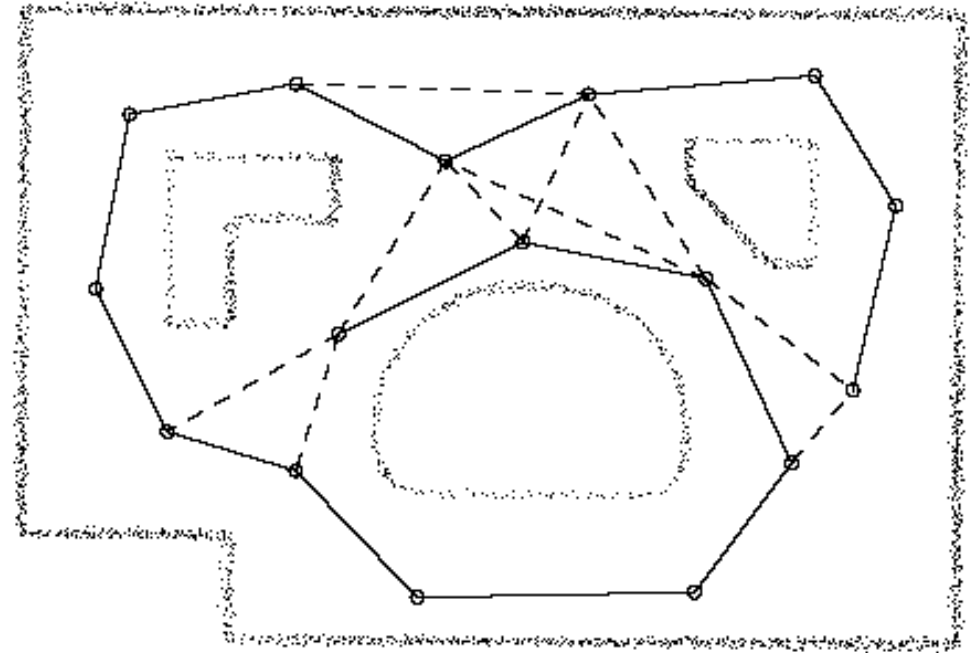
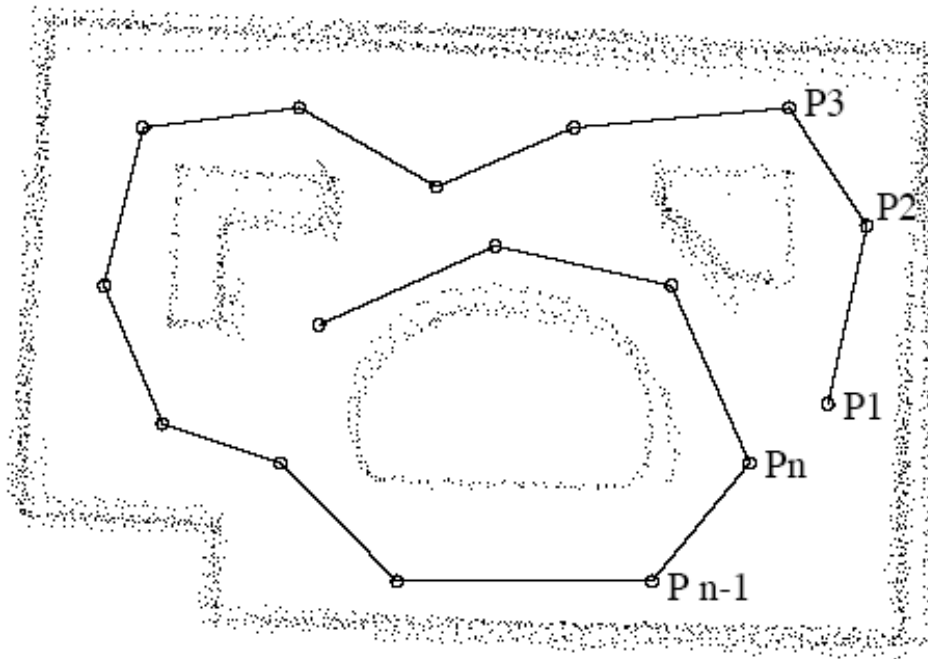
Video Sebastian Thrun,
robots.stanford.edu/videos.html



GraphSLAM - Die Idee



- ▶ Bei inkrementellem SLAM ist Pose-Historie nicht Teil des Zustands: Karte enthält nur Landmarken-Positionen
- ▶ **GraphSLAM** erweitert Zustandsraum um alle Posen
- ▶ Den wahrscheinlichsten Zustand (Vektor aus Posen & Landmarkenpositionen) schätze als den Vektor mit minimalem Abstand von den jeweiligen Erwartungswerten
- ▶ **Posefolge ist Teil der Karte!**



- ▶ Berücksichtige zusätzlich zur Odometrie (links) schlechte Scan-Matches in Scans mit großer Überlappung
- ▶ Maximiere die Wahrscheinlichkeit der Pose-Sequenz gegeben a-posteriori Fehler

F. Lu, E. Milios:
*Globally Consistent Range Scan Alignment for
Environment Mapping, Autonomous Robots, 1997*



Überlappung der Posen

- ▶ Als überlappend werden die Posen angesehen, die nahe am geschätzten Pfads des Roboters liegen oder eine signifikante Überlappung in den Sensordaten haben
- ▶ z.B. Überlappungsbereiche in Laserscans, Landmarken
- ▶ Der Registrierungsfehler zwischen sich überlappenden Posen wird als normalverteilte Zufallsvariable angenommen
- ▶ Die wahre Posedifferenz $D_{ij} = s_i - s_j$ zwischen zwei Knoten ist unbekannt
- ▶ Die beobachtete Posedifferenz ist $D'_{ij} = D_{ij} + \Delta D_{ij}$
- ▶ ΔD_{ij} ist normalverteilt mit Kovarianz C_{ij}
- ▶ Die Überlappung wird detektiert durch
 - Das Odometriemodell für benachbarte Posen
 - Transformation von überlappenden Umgebungsdaten (z.B. Scanmatching)



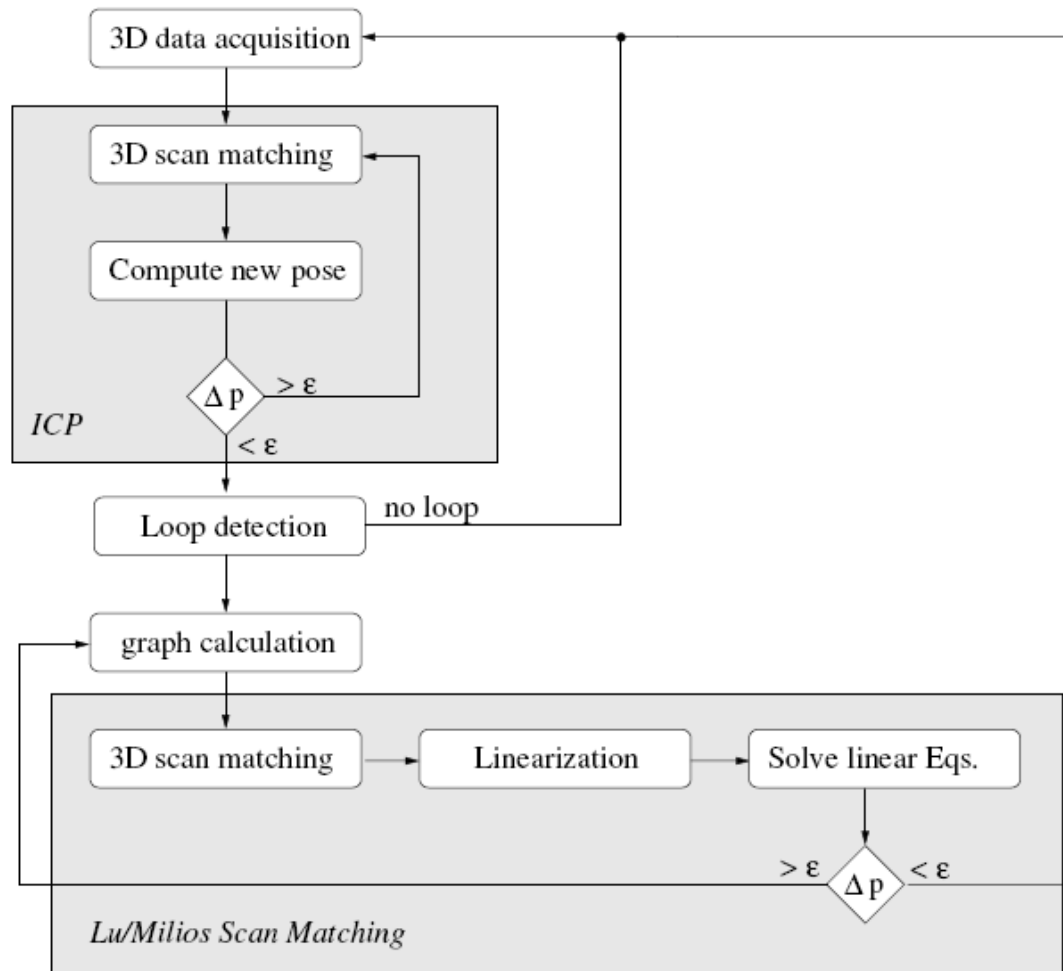
- ▶ Minimiere:

$$W = \sum_{(i,j), \text{overlapping}} (D_{ij} - D'_{ij})^T C_{ij}^{-1} (D_{ij} - D'_{ij})$$

- ▶ Die wahren D_{ij} und C_{ij} sind unbekannt
- ▶ Ausdruck ist nichtlinear (wegen der Rotation), aber eine Tayler-Approximation führt zu der quadratischen Form
- ▶ Für dieses Problem existieren geschlossene Lösungen:
 - 2D-Daten, 3D-Posen - Lu/Milios
 - 3D-Daten, 6D-Posen - Borrmann, Elseberg, Lingemann, Nüchter (JRAS 2008)

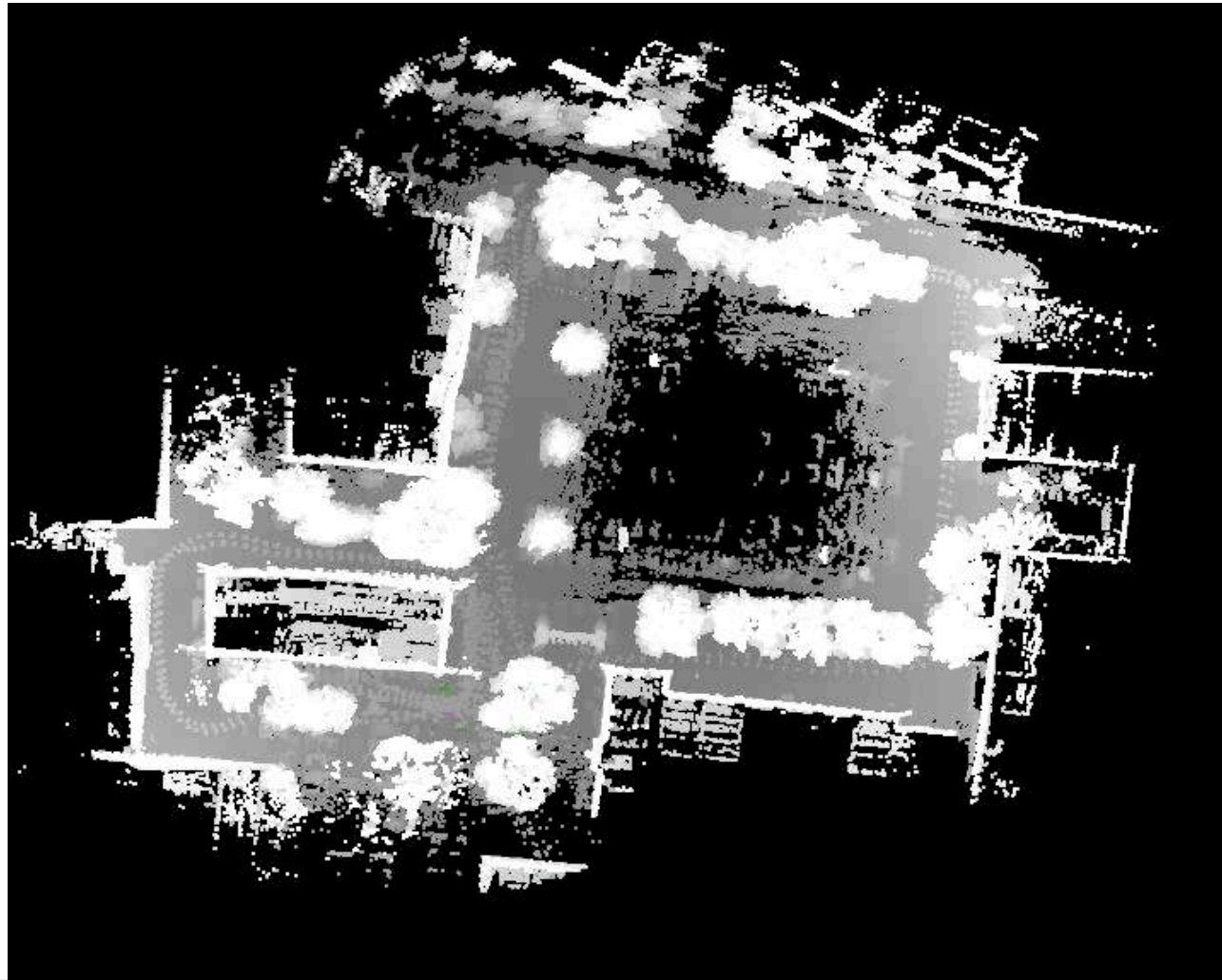


Schleifenschluss und globaler Ausgleich





6D ICP mit LUM





Gliederung

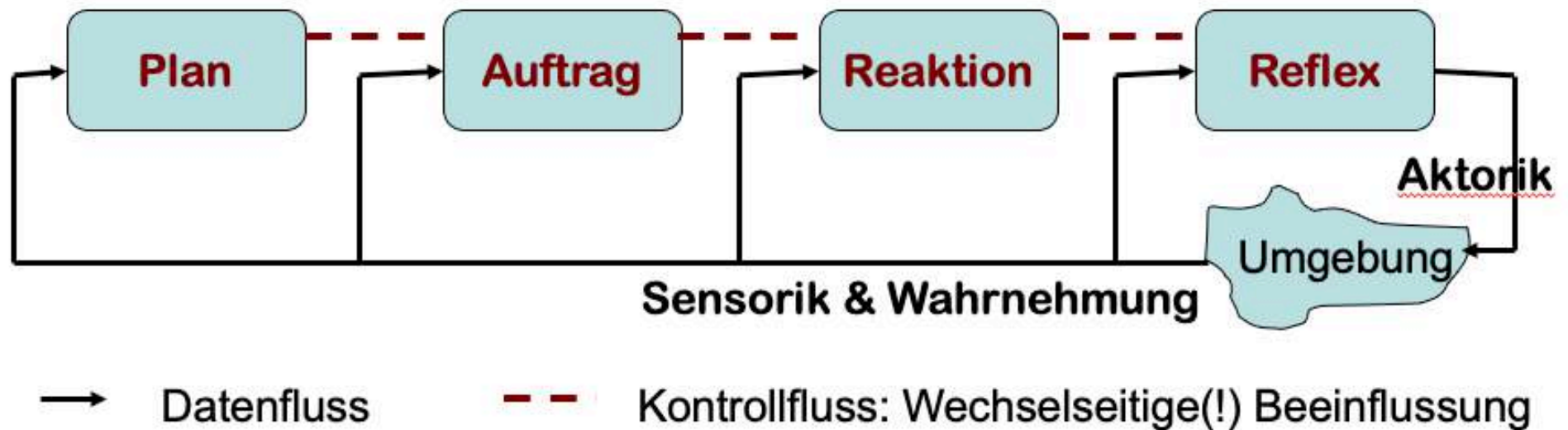
8. Navigation

1. **Hintergrund**
2. Reflexbasierte Navigation
3. Reaktive Navigation
4. Pfadplanung
5. Explorationsplanung
6. Mapping
7. Navigation
8. Ausblick



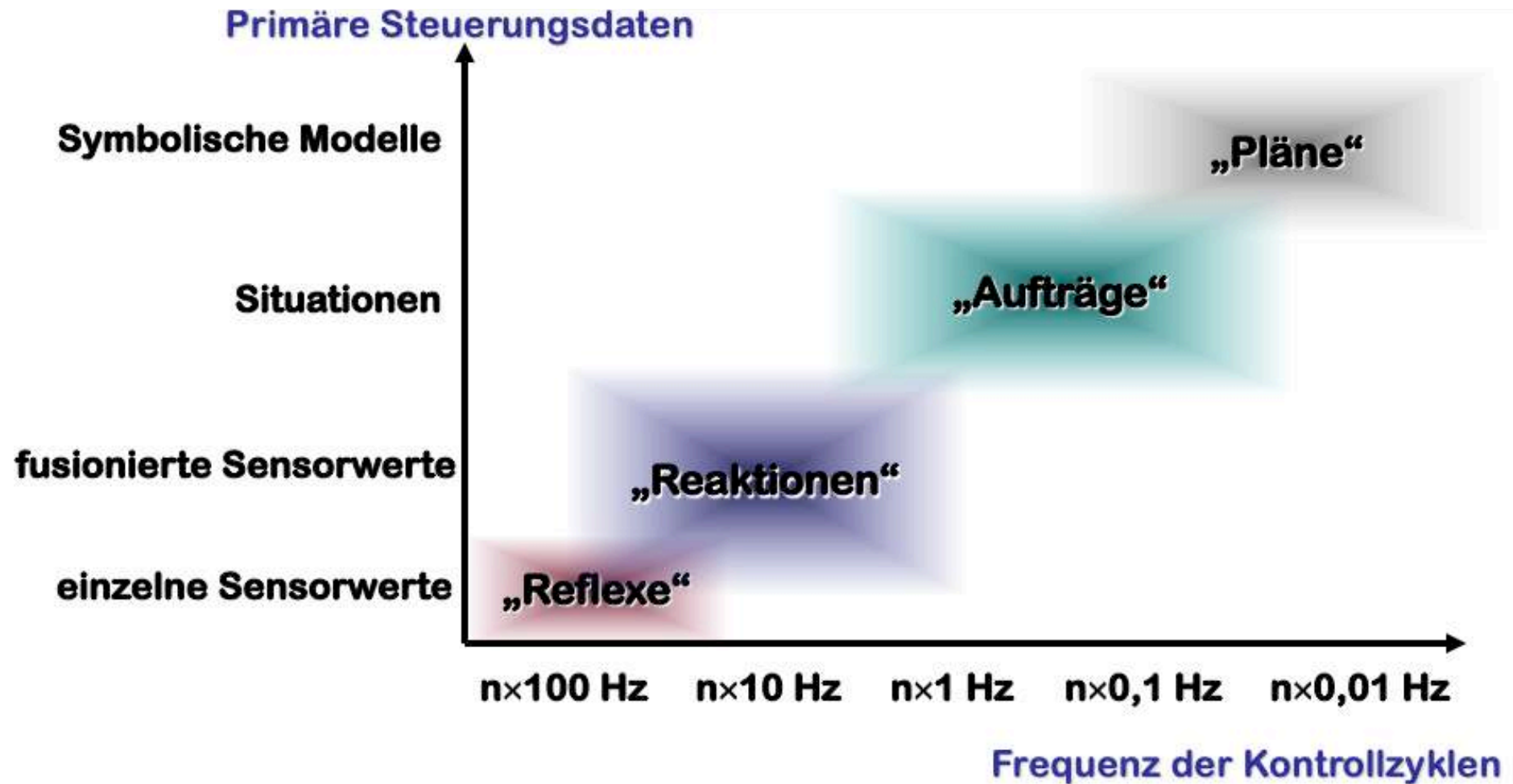
Modulkette zur Roboterkontrolle

- ▶ Zur Navigation (nicht nur dafür) ist starre Folge inadäquat!
- ▶ Sensormessen $\Rightarrow$ Interpretieren $\Rightarrow$ Schließen $\Rightarrow$ Handeln
- ▶ Grundmuster stattdessen: Modulkette



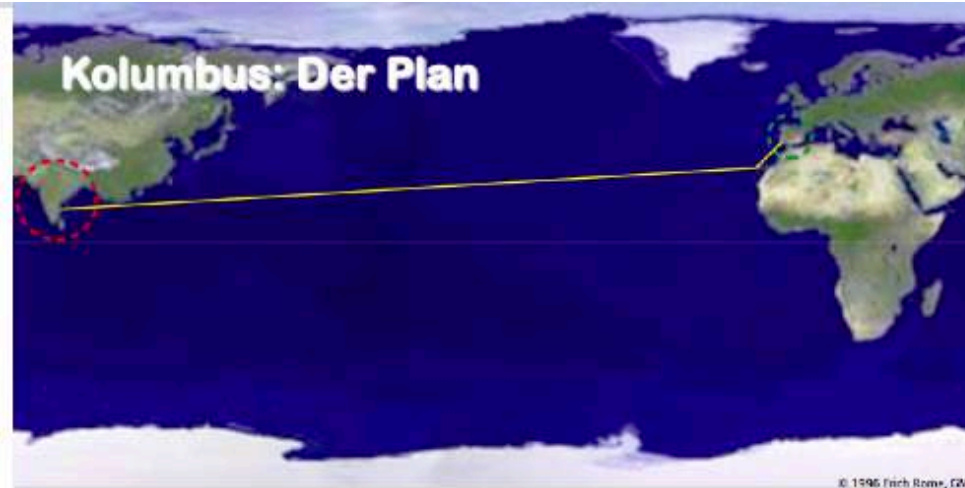


Zeitskalen und Granularitäten





Pläne sind wichtig...





Gliederung

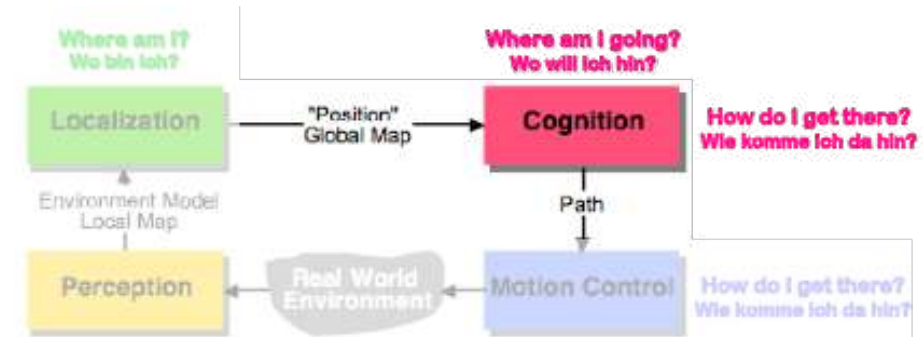
8. Navigation

1. Hintergrund
2. Reflexbasierte Navigation
3. Reaktive Navigation
4. **Pfadplanung**
5. Explorationsplanung
6. Mapping
7. Navigation
8. Ausblick



- ▶ Gegeben:
 - Karte der Umgebung (Geometrie, Belegungsraaster)
 - Startpose entspr. Karte
 - Zielposition oder -pose
- ▶ Finde den „besten“ Weg

- ▶ Ist übersetzt in Graphsuchproblem superklassisches Problem aus Informatik, KI (heuristische Suche!), OR, ...
 - Daher hier nur Abriss
 - Fokus auf Repräsentation d. Suchraums
 - Plane kompletten Pfad! Für Ausführung halte Zwischenzielpunkte



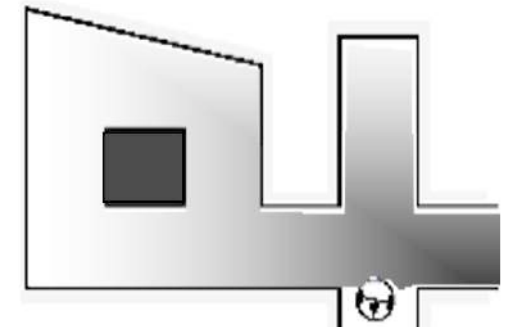
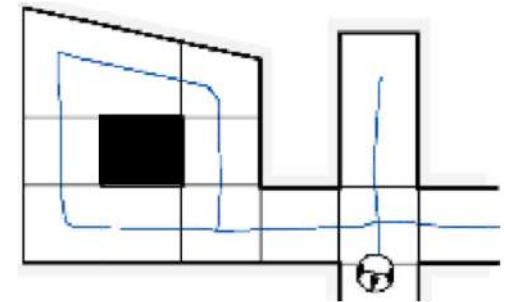
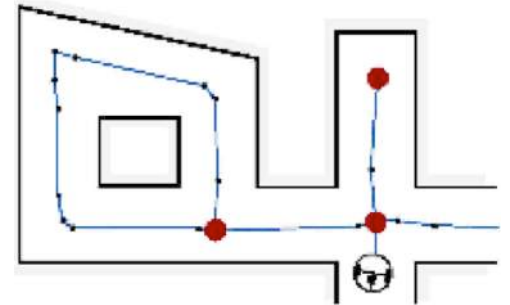
Jede Art Planung erfordert

- Information über Umgebung (evtl. nicht vollständig, nicht sicher)
- Stabilität, Gesetzmäßigkeit der Umgebung (evtl. nicht vollständig, nicht sicher)



Drei Familien von Pfadplanungsverfahren

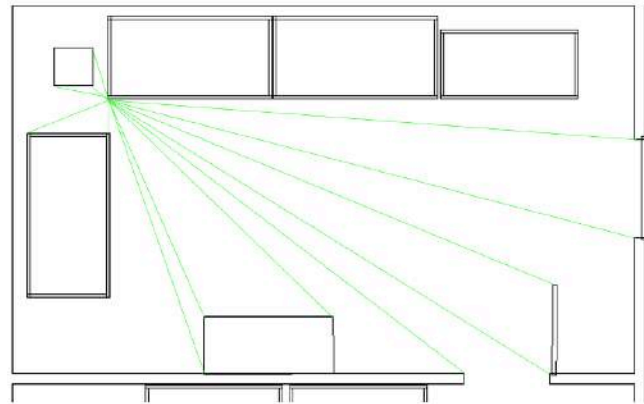
- ▶ **Straßenkarten** (“Road Maps”)
 - Identifiziere Wege im Freiraum
 - Wohin Zwischenpositionen?
 - Topologisch (merkmalbasiert), metrisch
- ▶ **Parkettierung** (“Cell Decomposition”):
 - Unterscheide freie vs unzugängliche Areale
 - Wohin Zellgrenzen?
 - Metrisch (Raster), topologisch (merkmalbasiert)
- ▶ **Potenzialfeld** (“Potential Fields”):
 - Überlagere Karte mit Gradienten („Nutzen“)





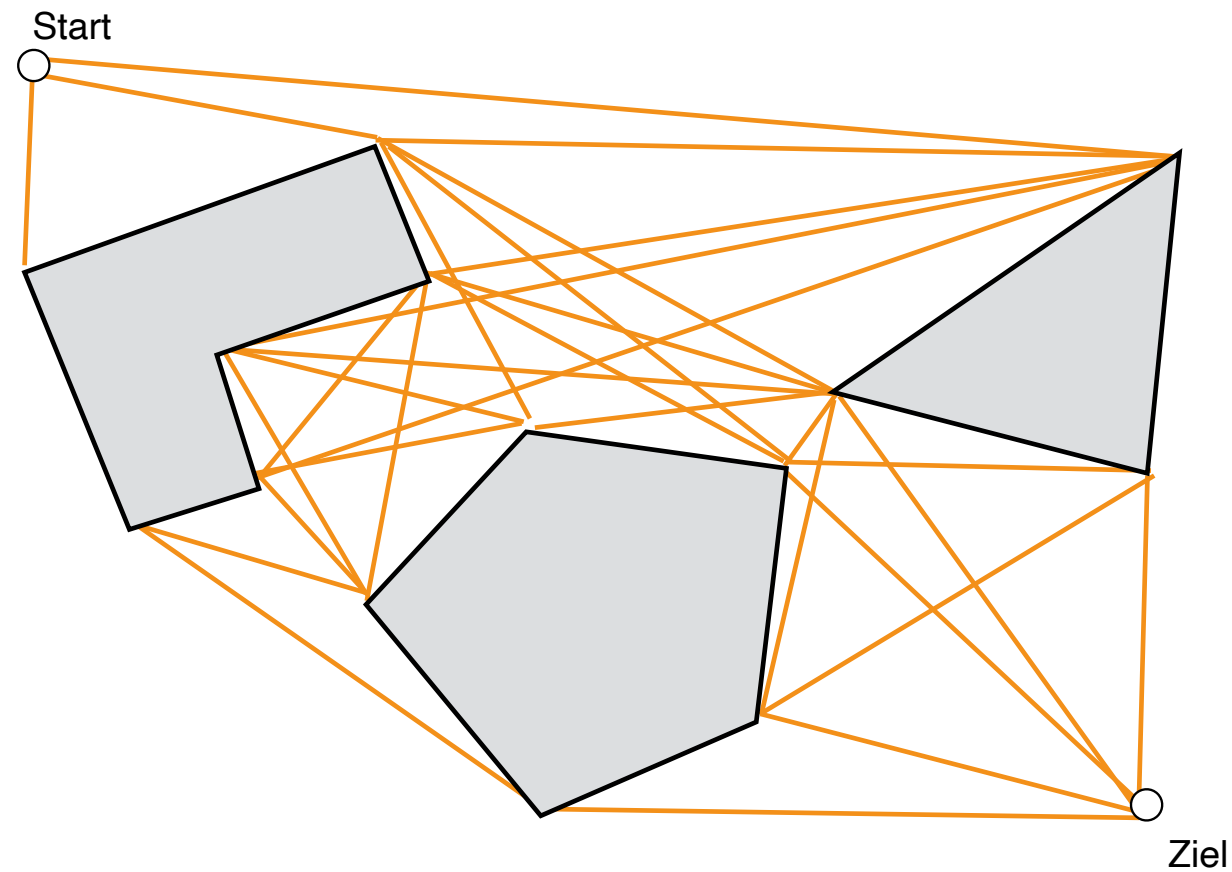
Straßenkarten 1 - Sichtbarkeitsgraphen

- **Knoten** sind alle Polygon-Ecken sowie Start- und Zielpunkt
 - Verbinde alle Knotenpunkte, die jeweils frei sichtbar sind
 - Suche kürzesten Weg (Summe der Streckenlängen) von Start nach Ziel entlang Sichtbarkeitskanten
-
- ➔ Graphkonstruktion $O(|Knoten|^2)$; Suche exponentiell i. Kanten
 - ➔ Findet geometrisch kürzesten Weg („Ideallinie“)
 - ➔ Weg ist schwierig zu fahren, kommt Hindernis-Ecken nah
 - ➔ Ausführung erfordert sichere Hindernisvermeidung
 - ➔ Tatsächlich gefahrener Weg möglicherweise nicht optimal kurz





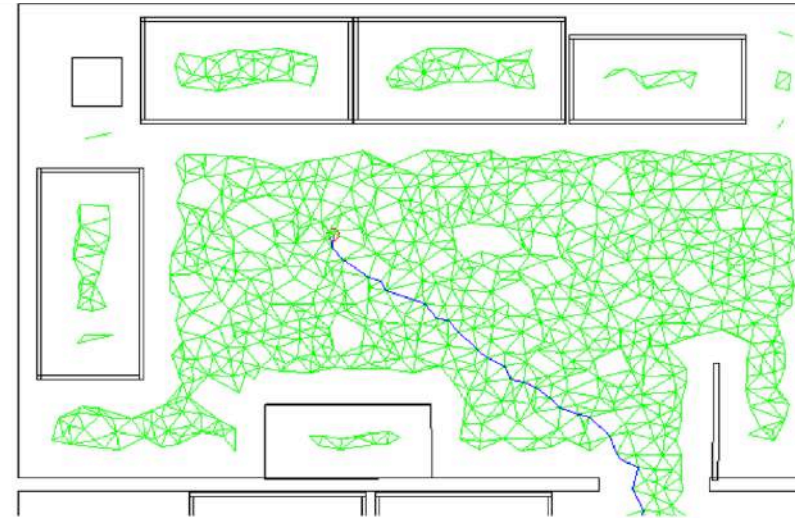
Sichtbarkeitsgraph (2)





Straßenkarten 2 - Probabilistisch

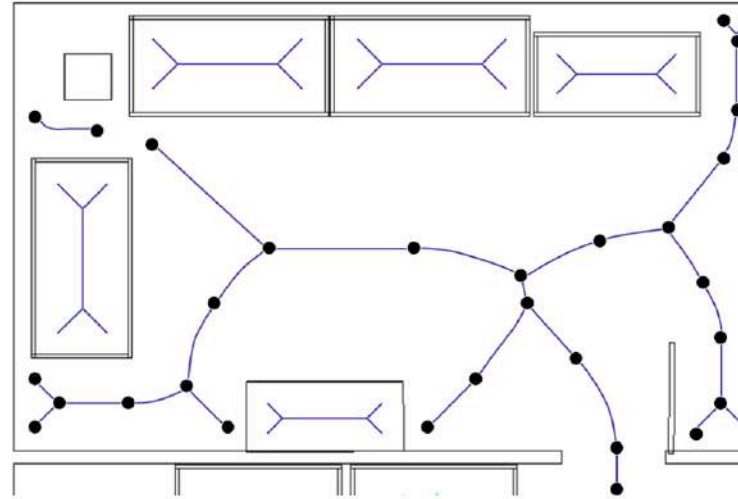
- Streue N Punkte gleichverteilt in den Freiraum + Start + Ziel
- Verbinde benachbarte Punkte durch planaren Graphen mit Kanten durch Freiraum
- Plane kürzesten Pfad entlang der Kanten
- ➔ Komplexität der Graph-Erzeugung und der Suche exponentiell in N , aber N wird überschaubar gewählt
- ➔ Pfad asymptotisch optimal; Hindernisabstand erhöht Länge
- ➔ Pfad kann unnötige „Zacken“ enthalten; ggf. glätten





Straßenkarten 3 - Voronoidiagramme

- **Voronoi-Graph:** Linien aus Punkten, die gleichen Abstand zu ihren beiden nächsten Hindernissen haben
- Platziere Knoten auf Kreuzungen und auf „lange“ Linien
- Verbinde Start und Ziel mit ihren nächsten Voronoi-Linien
- Suche kürzesten Weg im Graphen
 - ➔ Komplexität der Graphkonstruktion: $O(m \log m)$ für m Ecken
 - ➔ Pfade von suboptimaler Länge
 - ➔ Pfade maximieren Hindernisabstand, sind sicher fahrbar
 - ➔ bei zu großem Hindernisabstand ggf. Lokalisierungsprobleme





Parkettierung 2 - Rasterung

- Verwende Rasterung mit uniformer Zellengröße oder Quadrees wie eingeführt (Kartierung)
 - Verwende Zellmittelpunkte oder Mittelpunkte der Zellgrenzen als Suchknoten;
Kanten zwischen Knoten aus Nachbarzellen
-
- ➔ Rasterkarte gegeben (Kartierung) oder kostet $O(|\text{Zellen}|)$
 - ➔ Rasterung kann im Extremfall Wege blockieren
 - ➔ Pfadplanung nah-optimal bis auf Blockierung
 - ➔ Pfade durch Zellmitten/grenzen „zackelig“;
Fahrsteuerung glättet



Potenzialfeldverfahren

- Modelliere Zielpunkt durch „Anziehung“, Hindernisse durch „Abstoßung“
- Pfadplanung ist Gradientenabstieg im „Potenzialfeld“
- Für Punkt p :

$$U(p) = U_{\text{Ziel}}(p) + \sum U_{\text{Obj}}(p)$$

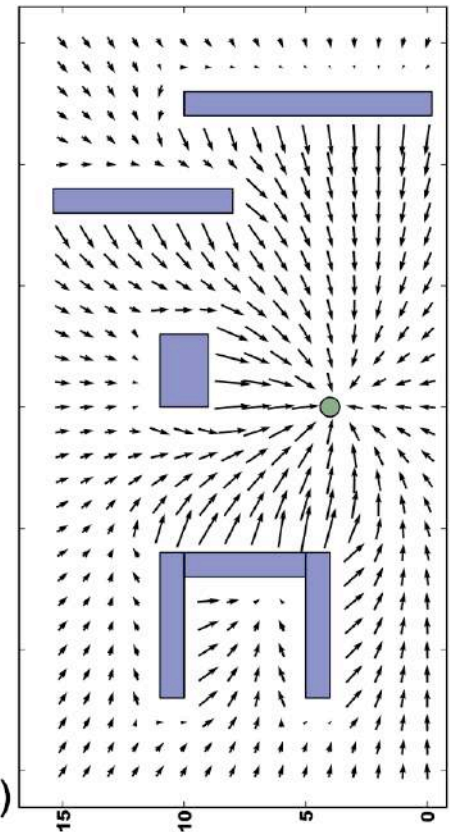
$$F = - \begin{bmatrix} \partial U / \partial x \\ \partial U / \partial z \end{bmatrix}$$

U_{Obj} meist
nur für nahe
Hindernisse
berücksichtigt

$$U_{\text{Ziel}}(p) = c_U \cdot \text{dist}(p, p_{\text{Ziel}})$$

$$U_{\text{Obj}}(p) = c_{\text{Obj}} \cdot \text{dist}(p, o)^{-2}$$

- ➔ Berechnung der Karte kann beliebig aufwändig sein
- ➔ Pfad kann in lokale Potenzialminima führen („Roboterfallen“)
- ➔ Pfade „elegant“



Diskretisierte Version (z.B. in Rasterzellen): **Vektorfelder**



- ▶ **Arbeitsraum** (“Work Space”): Der physische (geometrische, kartesisch modellierte) Raum, in dem d. Roboter sich bewegt
 - normalerweise 2D oder 3D
 - „das, was in einer Geometriekarte eingezeichnet ist“
- ▶ **Konfigurationsraum** (“Configuration Space”): Parameterraum, welcher mögliche Konfigurationen (Ort jedes Punktes des Roboterkörpers) des Roboters im Referenzsystem beschreibt
 - nicht eindeutig! (komplementäre K-Räume für 1 Roboter denkbar)
 - Pose (3D,6D) üblicher K-Raum für mobile Roboter
 - für Automations-Robotik üblich: K-Raum-Parameter = Freiheitsgrade

Ist #effektive Freiheitsgrade < Dimensionalität des K-Raums,
ist der Roboter nicht holonom!



Wahl eines Konfigurationsraums

- ▶ Gegeben Arbeitsraum, wie wählt man den passenden Konfigurationsraum?

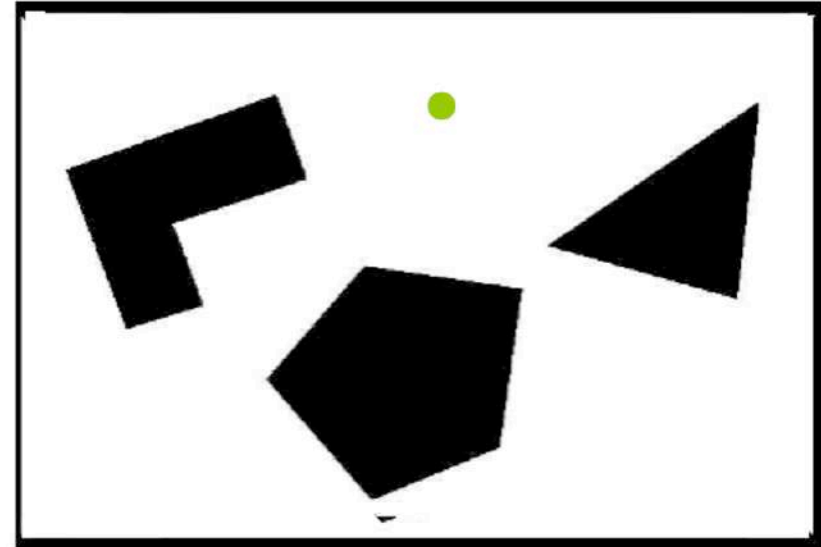
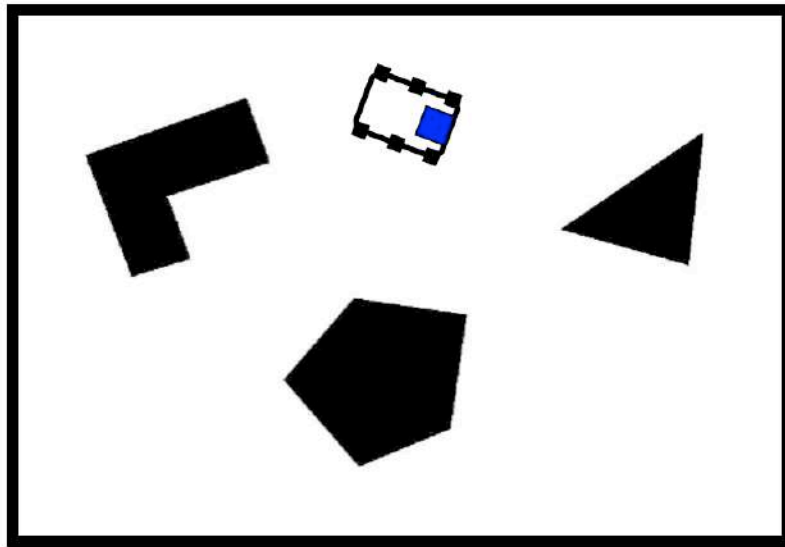
Es gibt nicht den K-Raum
zur Pfadplanung mobiler Roboter!

- ▶ manchmal unterbestimmt
 - (z.B. nicht-holonome Fahrzeuge: 2 DOF-KURT in 6-DOF-Pose Raum)
- ▶ manchmal überbestimmt
 - (z.B. redundante Kinematiken: 7DOF-Armroboter für 6DOF-Werkzeugpose)
- ▶ praktisch immer nichtlinear
 - (z.B. begrenzte Knickwinkel, K-Raum-Hindernisse)



Konfigurationsraum für 2D-Pfadplanung

- ▶ Geh aus von 2D-Karte mit polygonalen Objekten
- ▶ Vergrößere alle Objekte um halben Roboterdurchmesser
- ▶ Reduziere Roboter auf (holonomen) Punkt
- ▶ Als Approximation für Roboter, die auf der Stelle drehen können (reduziert 3D-Pose (x, z, θ) auf 2D K-Raum)

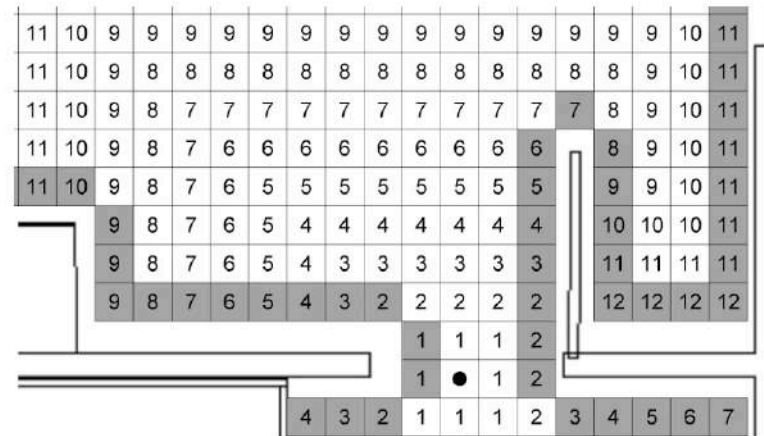




Wavefront-Planung

Falls oft zum selben Zielpunkt geplant werden muss:

- In Rasterkarte bewerte Zielzelle mit 0; trage rekursiv Kosten von Nachbarn bewerteter Zellen ein (Wavefront-Algorithmus)
- Ggf. bestrafe Zellen nahe Objekten (Straf-Term)
- Planung zu diesem Zielpunkt ist dann nicht mehr nötig: Folge von beliebigem Start dem Werte-Gradienten!



- ➔ Wavefront-Bewertung kostet $O(|\text{Zellen}|)$
- ➔ Pfadplanung nah-optimal
- ➔ Pfade wie bei (uniformen) Rasterkarten

Hybrid aus
Parkettierung und
Potenzialfeld



Ziele / Inhalte der Veranstaltung

